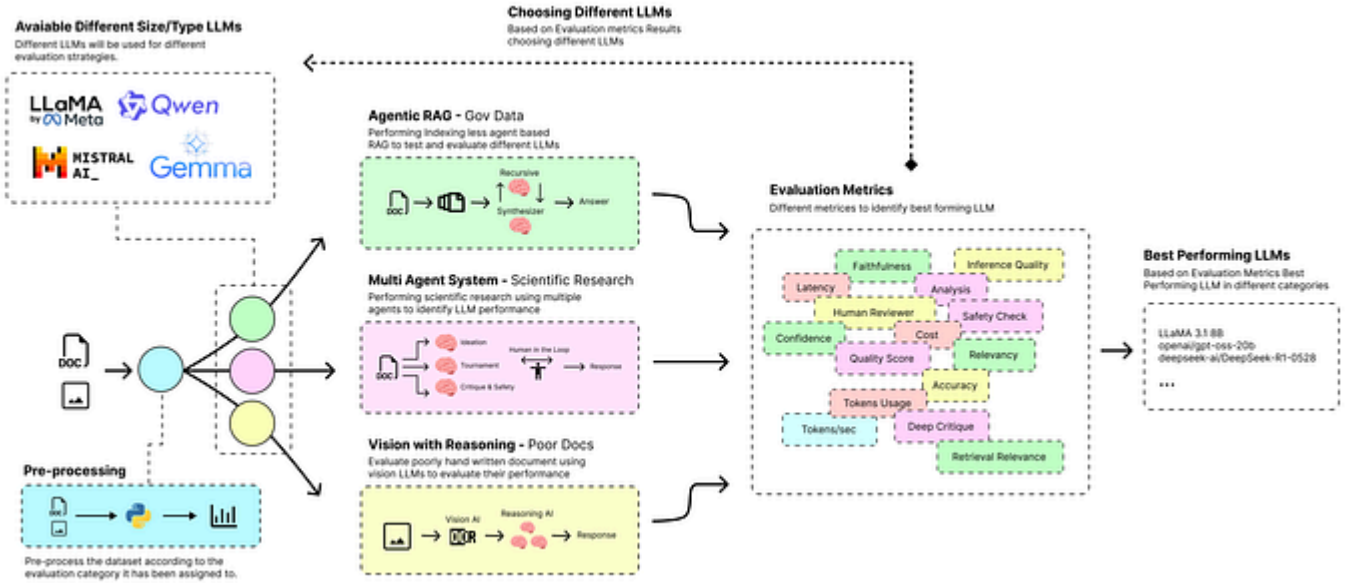


Building Three Pipelines to Select the Right LLMs for RAG, Multi-Agent Systems, and Vision

Agentic RAG, Multi-Agent and Vision with Reasoning

Fareed Khan

Level Up Coding · ~48 min read · August 22, 2025 (Updated: August 22, 2025) · Free: No

Read this story for free: link

In 2025, the most advanced systems no longer rely on a single large, general-purpose model. Instead, they use multiple LLMs of different sizes and specialties, each handling a specific task like a team of experts working together on a complex problem.

However, properly evaluating an LLM within its specific role in such an architecture is very challenging, **since it needs to be tested directly in the context where it is used.**

We are going to build three distinct, production-grade AI pipelines and use them as testbeds to measure how different models perform in their assigned roles. Our evaluation framework is built on three core principles:

1. **Role-Specific Testing:** We evaluate models based on the specific task they are assigned within the pipeline, whether it's a fast router, a deep reasoner, or a precise judge.
2. **Real-World Scenarios:** Instead of stopping at abstract tests, we are going to run our models through complete, end-to-end workflows to see how they

5. **Holistic Measurement:** We are going to measure everything that matters, from operational metrics like cost and latency to qualitative scores like faithfulness, relevance, and scientific validity.

All the code (theory + 3 notebooks) is available in my GitHub repository:

GitHub - FareedKhan-dev/best-llm-finder-pipeline: Agentic RAG, Multi-Agent Systems, and Vision...

Agentic RAG, Multi-Agent Systems, and Vision Reasoning are three pipelines to find the perfect LLM ...

github.com

Our codebase is organized as follows:

Copy

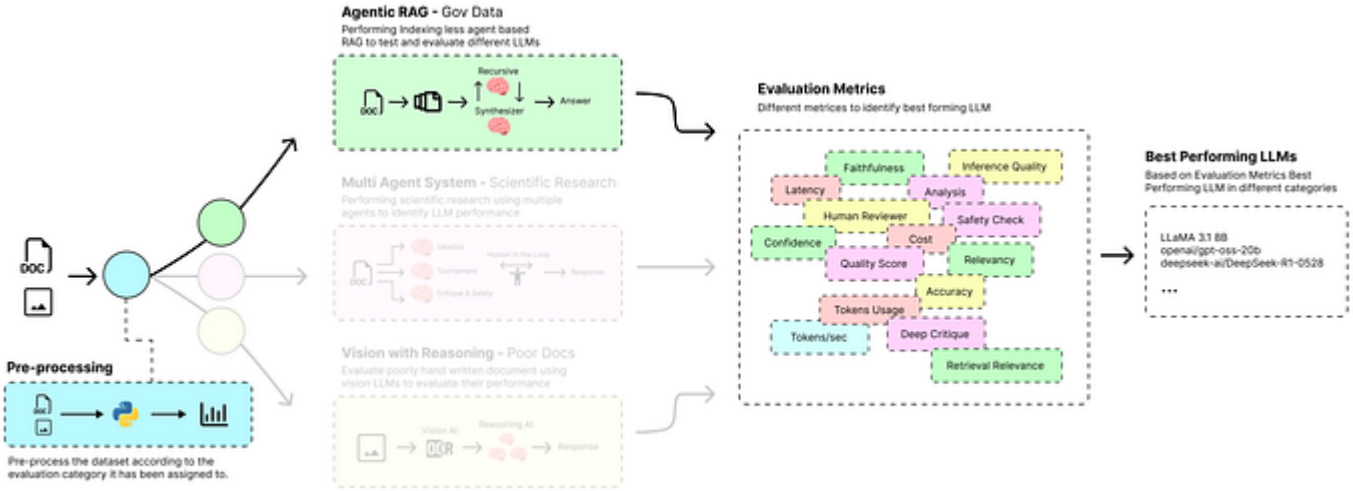
```
best-llm-finder-pipeline/
├── 01_agentic_RAG.ipynb        # Agentic RAG
├── 02_multi_agent_system.ipynb # Multi-agent system
├── 03_vision_reasoning.ipynb   # Vision reasoning
├── README.md                   # Project docs
└── requirements.txt            # Dependencies
```

Agentic RAG for Deep Document Analysis

Multi-Agent System for Scientific Research

Agentic RAG for Deep Document Analysis

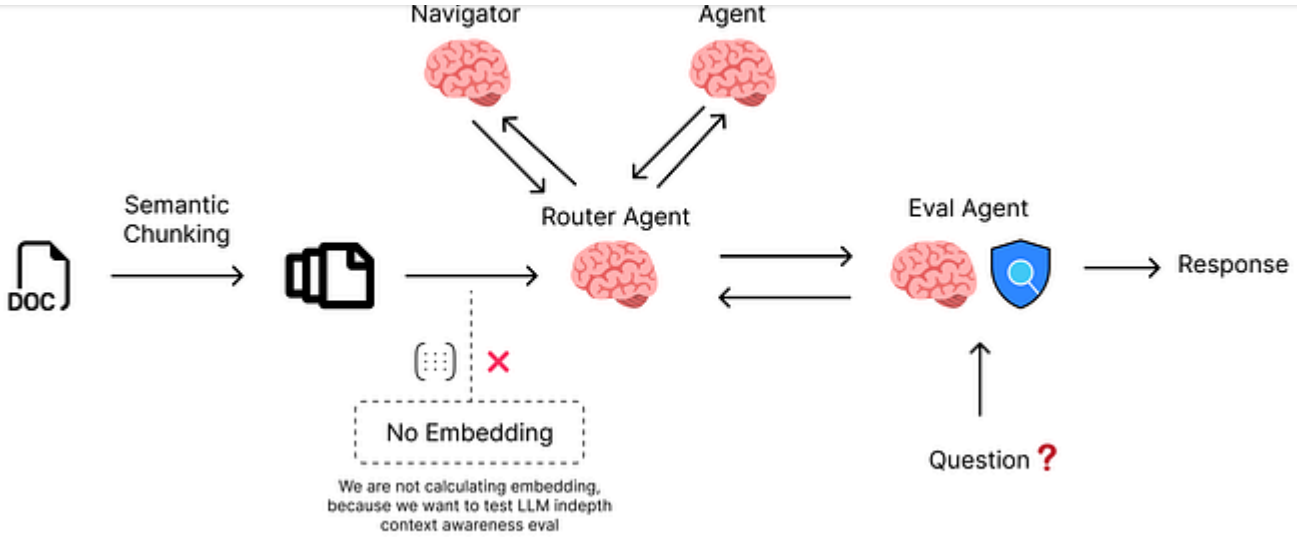
RAG systems based on agents are the new popular LLM-based solution in AI, especially for tackling the challenge of making sense of massive, unstructured



Agentic RAG (Created by Fareed Khan)

In this kind of architecture, LLMs are used in different parts to make the entire system work (**The correct way to evaluate LLMs**), from routing and retrieval to synthesis and evaluation.

> We'll avoid pre-indexing (like embedding-based RAG) and instead let an agent read documents on the fly, testing the LLM's depth and awareness.



Agentic RAG Pipeline (Created by Fareed Khan)

1. We start by loading a huge document and splitting it into a small number of large, high-level chunks. This creates a top-level map for the agent to navigate, avoiding a time-consuming pre-embedding step.
2. Then we create a **Router Agent**, based on a small and fast LLM, that skims these large chunks to identify the most relevant sections.
3. The system repeatedly takes the selected sections, splits them into smaller sub-chunks, and re-routes. This **zooming in** process allows the agent to progressively focus on the most promising information.

includes a precise citation back to its source.

5. Finally, we implement a **Evaluation Agent** that acts as an impartial judge, scoring the final answer for faithfulness and relevance to provide a quantitative confidence score along with other evaluation metrices.

Let's start building this pipeline.

Setting up the RAG Environment

Before we start coding, we need to import the necessary libraries that will be useful throughout this RAG pipeline.

Copy

```
import os                       # Interact with the operating system
import json                     # Work with JSON data
import re                       # Regular expression operations
import time                     # Time-related functions
from io import BytesIO          # Handle in-memory binary streams
from openai import OpenAI       # The OpenAI client library
from pypdf import PdfReader     # For reading and extracting text from PDF files
from typing import List, Dict, Any # Type hints for code clarity
```

another cloud service like Together AI.

Since I don't have a powerful local GPU, a cloud provider is the best way to access a range of models (They do offer free credits). Let's initialize our API client.

Copy

```
# --- LLM Configuration ---
# Replace with your API key and the base URL of your LLM provider
API_KEY = "YOUR_API_KEY" # Replace with your actual key
BASE_URL = "https://api.studio.nebius.com/v1/" # Or your Ollama/local host

# Initialize the OpenAI client to connect to the custom endpoint
client = OpenAI(
    api_key=API_KEY,
    base_url=BASE_URL,
)

# We will store performance metrics here throughout the run
metrics_log = []
```

We have also initialized an empty list, `metrics_log` , which will be important for our evaluation phase, as it will store detailed performance data for every LLM call we make.

A generic agentic RAG pipeline is often composed of at least three AI-driven components that we can optimize for cost and performance by choosing the right model for each.

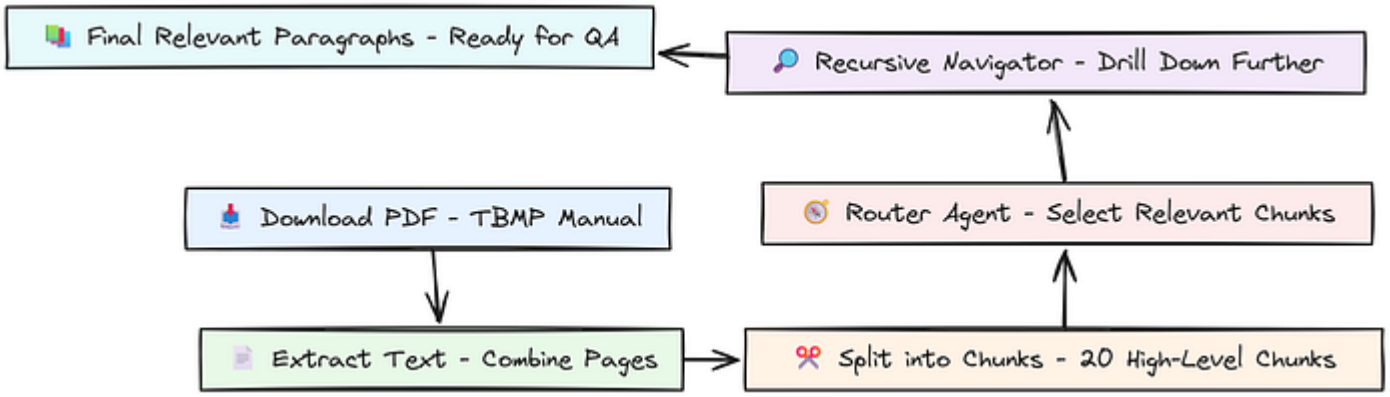
- **Routing:** This task involves skimming large amounts of text to find relevant sections. A smaller, faster model (like an 8B model) is often sufficient and highly cost-effective here.
- **Synthesis:** This task requires deep understanding and the ability to combine information from multiple sources into a coherent answer. A larger, more powerful model (like a 70B model) is a better choice.
- **Evaluation:** To reliably judge the quality of other models, we need a highly capable "judge" model. Using a top-tier model for this final check ensures our quality scores are trustworthy.

Copy

```
# Define the models to be used for different tasks
ROUTER_MODEL = "meta-llama/Meta-Llama-3.1-8B-Instruct"  # A fast model for rout
SYNTHESIS_MODEL = "meta-llama/Llama-3.3-70B-Instruct"   # A powerful model for
EVALUATION_MODEL = "deepseek-ai/DeepSeek-V3"       # A top-tier model for evalu
```

Preprocessing our Dataset

To properly evaluate our models, we need a challenging dataset. For this, we will use the **Trademark Trial and Appeal Board Manual of Procedure (TBMP)**, a dense, 920-page legal document. It's freely available to download, so let's write a function to fetch it and extract its text.



Pre-processing Step (Created by Fareed Khan)

Copy

```
# URL for the TBMP manual
tbmp_url = "https://www.uspto.gov/sites/default/files/documents/tbmp-Master-Jur

# Maximum number of pages to process
```

```
# Download the PDF file from the URL
response = requests.get(tbmp_url)
response.raise_for_status()  # Check if the download was successful

# Read the PDF content from the response
pdf_file = BytesIO(response.content)
pdf_reader = PdfReader(pdf_file)

# Determine the number of pages to process
num_pages_to_process = min(max_pages, len(pdf_reader.pages))

full_text = ""
# Loop through pages
for page in pdf_reader.pages[:num_pages_to_process]:
    # Extract text from the current page
    page_text = page.extract_text()
    if page_text:
        # Append page text to the full document string
        full_text += page_text + "\n"

document_text = full_text
```

We are targeting the first 920 pages of the doc and appending each page extracted text into a single variable which will result in one large string, let's print the sample and some other info.

```
# Display document statistics
char_count = len(document_text)
token_count = count_tokens(document_text)
print(f"\nDocument loaded successfully.")
print(f"- Total Characters: {char_count:,}")
print(f"- Estimated Tokens: {token_count:,}")

# Show a preview of the extracted text
print("\n--- Document Preview (first 500 characters) ---")
print(document_text[:500])


#### OUTPUT ####
Document loaded successfully.
- Total Characters: 3,459,491
- Estimated Tokens: 945,084

--- Document Preview (first 500 characters) ---
June   2024
United States Patent and Trademark Office
PREFACE TO THE JUNE 2024 REVISION
The June 2024 revision of the Trademark Trial and Appeal Board Manual of Proced
June 2023 edition. This update is moderate in nature and incorporates relevant
3, 2023 and March 1, 2024.
The title of the manual is abbreviated as "TBMP." A citation to a section of th
...
```

create a small number of large, high-level chunks (e.g., 20). This forms the top level of our document hierarchy.

Copy

```
def split_text_into_chunks(text: str, num_chunks: int = 20) -> List[Dict[str, A
    """
    Splits a long text into a specified number of chunks, respecting sentence b
    """
    # First, split the entire text into individual sentences
    sentences = sent_tokenize(text)
    if not sentences:
        return []

    # Calculate how many sentences should go into each chunk on average
    sentences_per_chunk = (len(sentences) + num_chunks - 1) // num_chunks

    chunks = [] # Initialize list to hold the chunks
    # Set progress bar description, only for long texts
    desc = "Creating chunks" if len(sentences) > 500 else None
    # Iterate over the sentences list in steps of sentences_per_chunk
    for i in tqdm(range(0, len(sentences), sentences_per_chunk), desc=desc):
        # Slice the sentences list to get the sentences for the current chunk
        chunk_sentences = sentences[i:i + sentences_per_chunk]
        # Join the sentences back into a single string
        chunk_text = " ".join(chunk_sentences)
        # Append the new chunk as a dictionary to the list
```

```
            text": chunk_text
        })

    # Print a confirmation message with the number of chunks created
    print(f"Document split into {len(chunks)} chunks.")
    return chunks
```

We are going for chunking that is designed to be sentence-aware, which means that it will try to avoid splitting sentences in the middle. This helps preserve the semantic integrity of the text.

Let's run this chunking on our document and check the different chunks along with the total number of tokens.

Copy

```
# Split the document text into a specified number of chunks.
document_chunks = split_text_into_chunks(document_text, num_chunks=20)

# Display stats for the first few chunks.
for chunk in document_chunks[:3]:
    # Count the tokens in the current chunk's text.
    chunk_token_count = count_tokens(chunk['text'])
    # Print the chunk ID and its token count.
    print(f"- Chunk {chunk['id']}: {chunk_token_count:,} tokens")
```

```
Document split into 20 chunks:
- Chunk 0: 42,822 tokens
- Chunk 1: 42,367 tokens
- Chunk 2: 42,516 tokens
```

As you can see, we have 20 massive chunks, each around 42k tokens. This is a deliberate choice. These chunks are far too large for a final answer, but they provide excellent high-level context for our Router Agent to make its initial broad selection.

The size is also designed to be well within the context window of our chosen `ROUTER_MODEL` , Llama 3.1 8B, which supports up to 128k tokens.

With our top-level chunks ready, we can now build the core of our system.

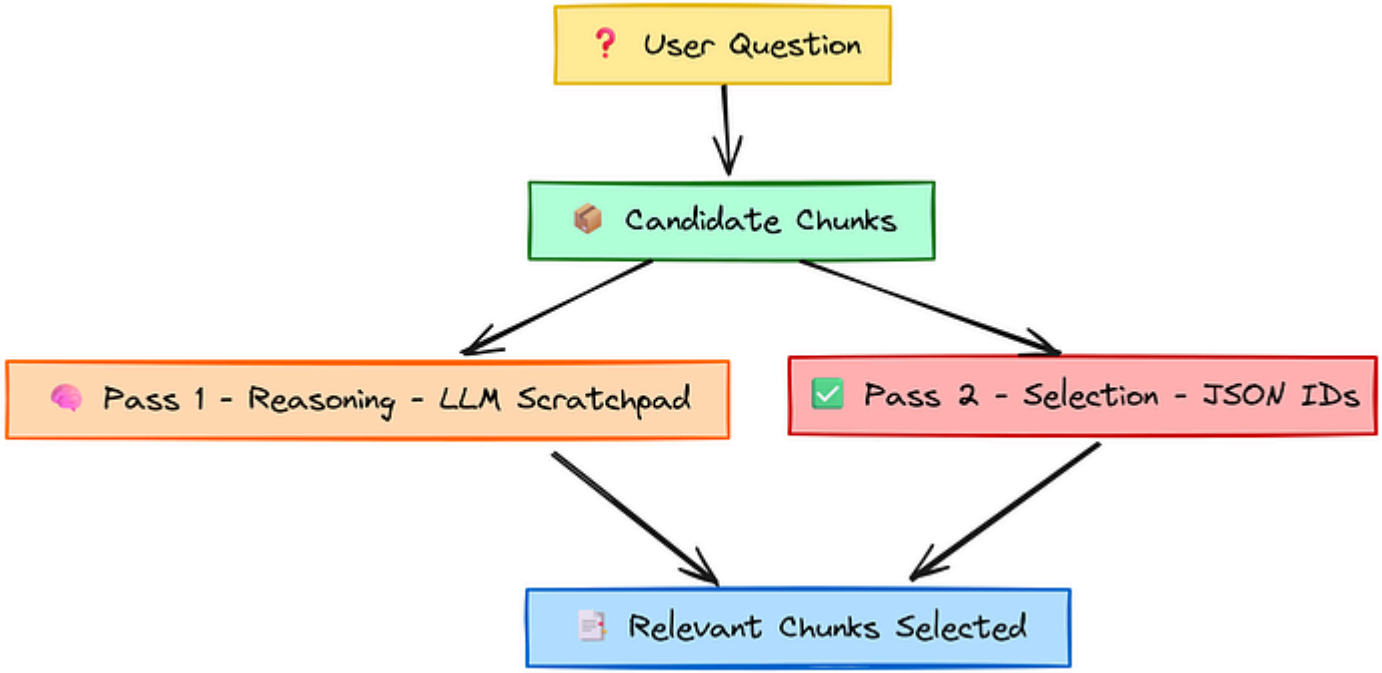
> Instead of embedding everything, we will build a system that mimics how a human researcher would tackle this document.

1. It starts by skimming the high-level chunks (our 20 chunks) to find relevant sections.
2. Then reads those sections more closely by breaking them into smaller sub-chunks, and repeats this process until it has isolated a handful of highly

This entire process is managed by a recursive loop, which will be supported by two key components we will build next: **Router Agent** to select relevant chunks and a **Recursive Navigator** to manage the drill-down process.

Creating the Two-Pass Router Agent

The **brain** of our navigation system is the Router Agent. Its job is to look at a set of text chunks and decide which ones are relevant to the user's question.



Two Pass Router (Created by Fareed Khan)

1. **Reasoning Pass:** First, we ask the LLM to review the chunks and write down its reasoning in a **scratchpad**. This is like an internal monologue where it thinks about which chunks seem promising and why.
2. **Selection Pass:** Second, we take its reasoning and ask it to make a final, structured decision, returning only the IDs of the selected chunks.

This separation forces a more deliberate thought process and consistently leads to better, more accurate routing. Before we build the agent, we need a robust helper function for parsing the JSON it will output.

> LLMs can sometimes be inconsistent, so this utility makes our system more resilient.

Copy

```
def parse_json_from_response(text: str) -> Dict[str, Any]:
    """
    Extracts and parses a JSON object from a string, even if it's embedded in r
    """
    # Search for a JSON block within markdown ```json ... ```
    match = re.search(r'```(?:json)?\\s*({.*?})\\s*```', text, re.S)
    if match:
        json_str = match.group(1)
    else:
```

```
        end = text.rfind('}')
        if start != -1 and end != -1:
            json_str = text[start:end+1]
        else:
            # If no JSON structure is found, this is a fallback
            return {} # Return empty dict if no JSON is found

    try:
        # Try to parse the extracted string as JSON
        return json.loads(json_str)
    except json.JSONDecodeError:
        # If parsing fails, log a warning and return an empty dictionary
        print(f"Warning: Failed to parse JSON from response. Raw text: '{text}
        return {}
```

It's very simple logic here, `parse_json_from_response` function acts as our universal JSON parser. It first tries to find a JSON object wrapped in markdown code fences (```` ```json ... ``` ````). If that fails, it looks for the first opening brace `{` and the last closing brace `}` to isolate the JSON string.

Now, let's build the router agent function itself.

Copy

```
def route_to_chunks(question: str, chunks: List[Dict[str, Any]], scratchpad: st
    """
```

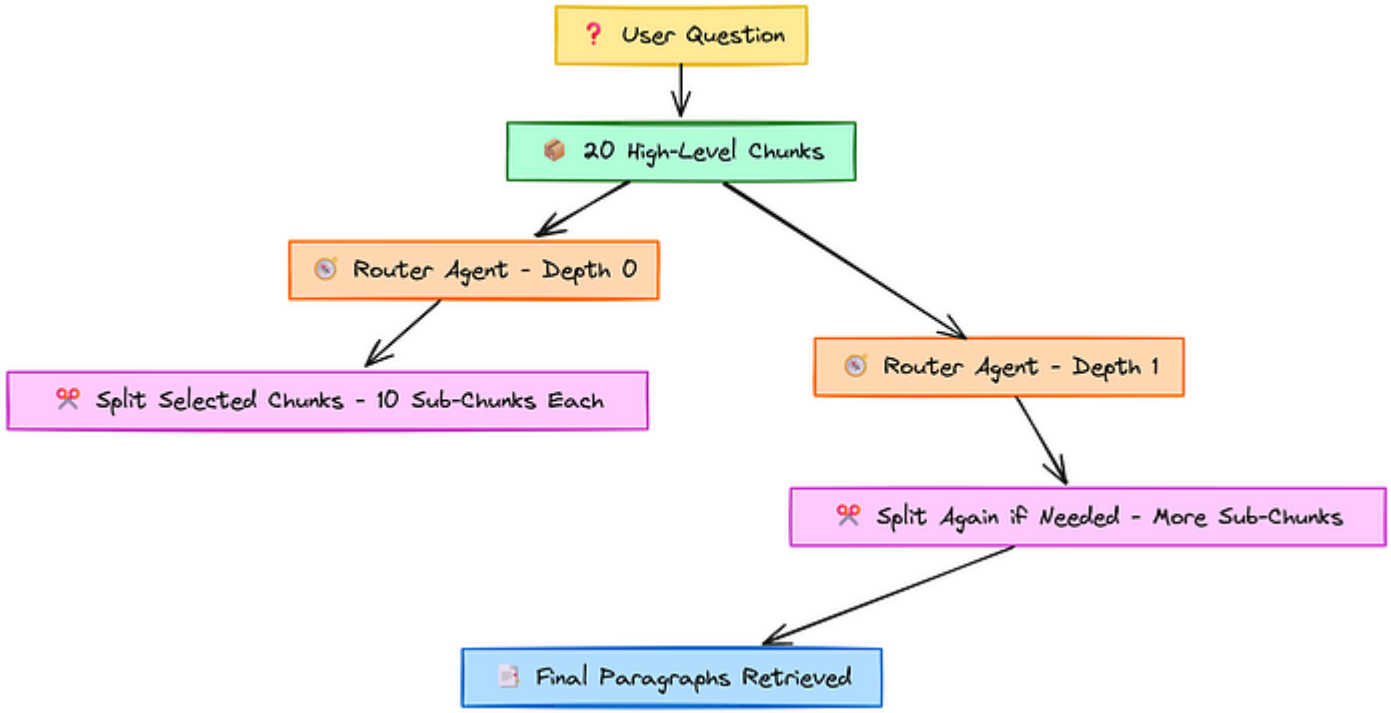
```
print(f"\n--- Routing at Depth {depth}. Evaluating {len(chunks)} chunks ---

# Format the chunks for the prompt, showing only the first 1000 characters
chunks_formatted = "\n\n".join([f"CHUNK {chunk['id']}:\n{chunk['text'][:100

# PASS 1: Generate reasoning
reasoning_prompt = f"""
You are an expert document analyst. Your goal is to find information to ans
'{question}'

Here is your reasoning so far:
{scratchpad}

Review the following new text chunks. Briefly explain which chunks seem rel

TEXT CHUNKS:
{chunks_formatted}

Your Reasoning:
"""

reasoning_response = client.chat.completions.create(model=ROUTER_MODEL, mes
new_reasoning = reasoning_response.choices[0].message.content

# Update the scratchpad with the new reasoning for the next level of depth
updated_scratchpad = scratchpad + f"\n[Depth {depth} Reasoning]: {new_reasc
print(f"LLM Reasoning: {new_reasoning}")

# PASS 2: Make the final selection based on the reasoning
```

```
    Your Reasoning:
    {new_reasoning}

    TEXT CHUNKS:
    {chunks_formatted}

    Respond with ONLY a valid JSON object with a single key 'selected_chunk_ids
    """

    selection_response = client.chat.completions.create(model=ROUTER_MODEL, mes
    response_text = selection_response.choices[0].message.content

    # Parse the final JSON output and extract the selected IDs
    parsed_output = parse_json_from_response(response_text)
    selected_ids = parsed_output.get('selected_chunk_ids', [])
    print(f"Selected chunk IDs: {selected_ids}")

    return {"selected_ids": selected_ids, "scratchpad": updated_scratchpad}
```

The following `route_to_chunks` function implements our two-pass logic.

1. It formats the provided chunks into a concise prompt, orchestrates the two sequential API calls to the `ROUTER_MODEL`, and ...
2. returns the final list of selected chunk IDs along with the updated reasoning scratchpad.

Next comes the Recursive Navigator, which will manage the entire drill-down process, calling the Router Agent at each level.



Recursive Navigator (Created by Fareed Khan)

1. It starts with our 20 top-level chunks. After the Router Agent selects a few, the Navigator takes each selected chunk, splits it into smaller sub-chunks (e.g., 10), and then presents these new, more detailed sub-chunks to the Router Agent in the next iteration.

A key feature is that it tracks the **path** of each chunk (e.g., `9.0.4`), which will be essential for providing precise citations later.

Copy

```
def navigate_document(question: str, initial_chunks: List[Dict[str, Any]], max_
    """
    Performs a hierarchical navigation of the document to find relevant paragra
    """
    scratchpad = ""
    current_chunks = initial_chunks
    final_paragraphs = []

    # Keep track of the hierarchical path of each chunk for citation purposes
    chunk_paths = {chunk["id"]: str(chunk["id"]) for chunk in initial_chunks}

    # Loop through the desired depth of navigation
    for depth in tqdm(range(max_depth), desc="Navigating Document"):
        # Call the router to select relevant chunks at the current level
        result = route_to_chunks(question, current_chunks, scratchpad, depth)
        scratchpad = result["scratchpad"]
        selected_ids = result["selected_ids"]

        # If the router returns no selections, we stop the navigation early
        if not selected_ids:
            print("\nNavigation stopped: No relevant chunks selected.")
```

```
        # Filter to get the full text of the selected chunks
        selected_chunks = [c for c in current_chunks if c["id"] in selected_ids

        # Prepare the next level of chunks by splitting the selected ones
        next_level_chunks = []
        next_chunk_id_counter = 0
        for chunk in selected_chunks:
            parent_path = chunk_paths[chunk["id"]]
            # Split the selected chunk into 10 smaller sub-chunks
            sub_chunks = split_text_into_chunks(chunk['text'], num_chunks=10)

            # Assign new IDs and create the hierarchical path for each sub-chur
            for i, sub_chunk in enumerate(sub_chunks):
                new_id = next_chunk_id_counter
                sub_chunk['id'] = new_id
                chunk_paths[new_id] = f"{parent_path}.{i}"
                next_level_chunks.append(sub_chunk)
                next_chunk_id_counter += 1

        current_chunks = next_level_chunks
        final_paragraphs = current_chunks # Update the final paragraphs at each

    print(f"\nNavigation finished. Returning {len(final_paragraphs)} retrieved
    # Add the final display_id to each paragraph for easy citation
    for chunk in final_paragraphs:
        if chunk['id'] in chunk_paths:
             chunk['display_id'] = chunk_paths[chunk['id']]
```

Our `navigate_document` function sets up the main loop that runs for `max_depth` iterations.

1. In each cycle, it calls our `route_to_chunks` agent, processes the results, and prepares the next, more granular level of sub-chunks for the next cycle.
2. It tracks the path of each chunk (e.g., `9.0.4`) to build a trail for our final citations.

Executing the Full Navigation Process

With all the components we have built, let's run the entire navigation process.

> We'll ask a specific legal question and set the navigation depth to 2.

This will trigger a series of LLM calls as the agent first skims the high-level document and then drills down into the most promising section.

Copy

```
# The legal question we want to answer from the manual
sample_question = "What are the requirements for filing a motion to compel disc
```

```
navigation_result = navigate_document(sample_question, document_chunks, max_dep

print(f"\n--- Navigation Complete ---")
print(f"Retrieved {len(navigation_result['paragraphs'])} paragraphs for synthes

# Display a preview of the first retrieved paragraph to see the result
if navigation_result['paragraphs']:
    first_para = navigation_result['paragraphs'][0]
    print(f"\n--- Preview of Retrieved Paragraph {first_para.get('display_id',
    print(first_para['text'][:500] + "...")
    print("----------------------------------------")
```

When we run this navigation flow this is what we get.

Copy

```
--- Routing at Depth 0: Evaluating 20 chunks ---
LLM Reasoning: After reviewing the provided text chunks, CHUNK 9 seems most re
Selected chunk IDs: [9]

Document split into 10 chunks.

--- Routing at Depth 1: Evaluating 10 chunks ---
LLM Reasoning: Based on the new text chunks from the original CHUNK 9, I belie
Selected chunk IDs: [0, 4, 8]

Document split into 10 chunks.
Document split into 10 chunks.
```

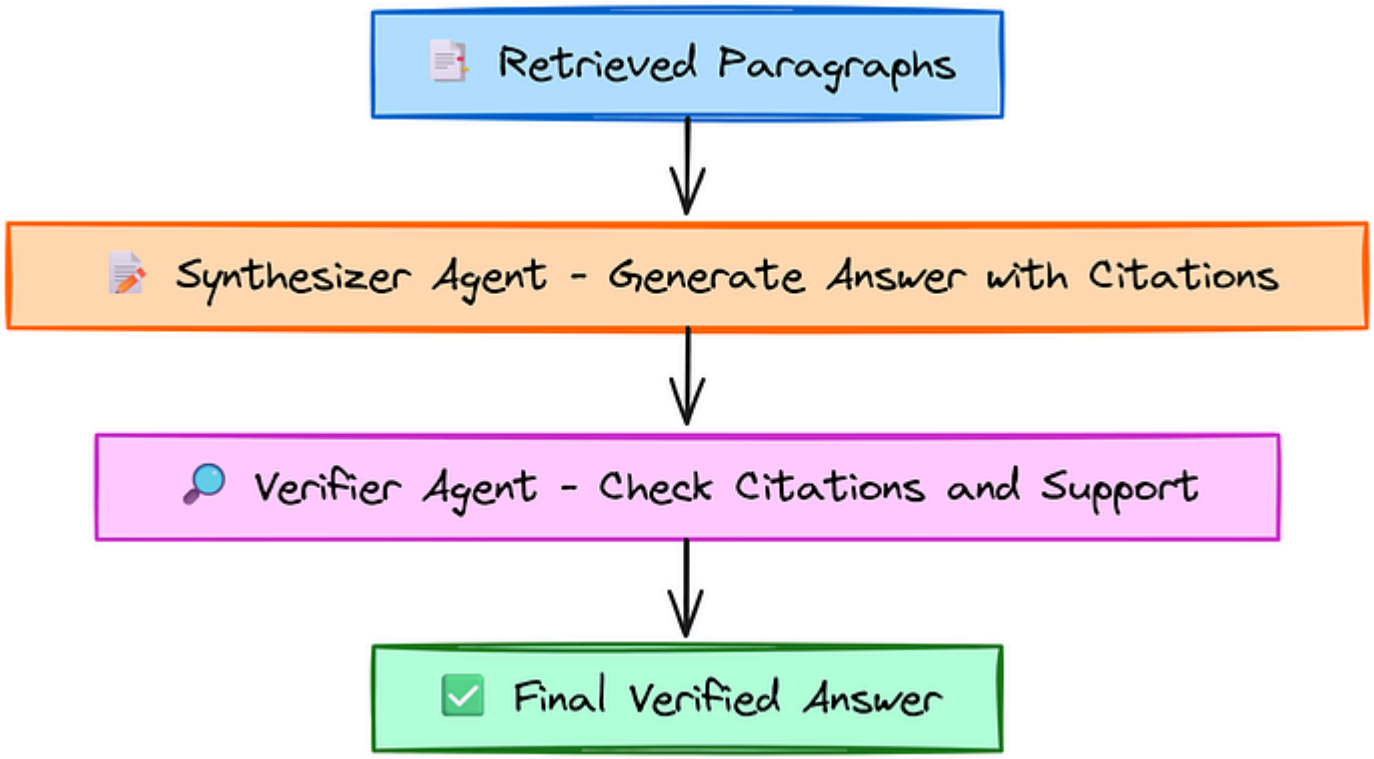
```
Navigation finished. Returning 30 retrieved paragraphs.

--- Navigation Complete ---
Retrieved 30 paragraphs for synthesis.

--- Preview of Retrieved Paragraph 9.0.0 ---
See MISCELLANEOUS CHANGES TO TRADEMARK TRIAL AND APPEAL
BOARD RULES OF PRACTICE, 81 Fed. Reg. 69950, 69951, 69977 (October 7, 2016). 3.
TRIAL AND APPEAL BOARD RULES, 72 Fed. Re g. 42242, 42256 (August 1, 2007) (" A
sanctions is only appropriate if a motion to compel these respective disclosure
Amazon Technologies v. Wax , 93 USPQ2d 1702, 1706 (TTAB 2009) (motion for sa...
----------------------------------------
```

The output shows the agent thought process.

- At **Depth 0**, it analyzed all 20 high-level chunks and correctly identified `CHUNK 9` as the most relevant one for our query.
- It then split `CHUNK 9` into 10 smaller sub-chunks.
- At **Depth 1**, it analyzed these 10 sub-chunks and further narrowed its focus to sub-chunks `0`, `4`, and `8`.

The process has drilled down from a nearly one-million-token document to a curated set of 30 highly relevant paragraphs. This targeted context is now ready for our final Synthesizer agent.

After our navigation agent has successfully drilled down and retrieved a set of highly relevant paragraphs. We now have the **raw material**, but we need a specialist to craft it into a final product. This is the role of the **Synthesizer Agent.**



Citations Check and Verify Agent (Created by Fareed Khan)

instruct this agent to adhere to two strict rules:

1. **Grounding:** It must base its answer *only* on the information contained within the provided paragraphs, preventing any hallucinations or use of external knowledge.
2. **Traceability:** For every statement it makes, it must cite the specific paragraph ID it came from. This creates a fully verifiable and trustworthy answer.

Copy

```
def generate_answer(question: str, paragraphs: List[Dict[str, Any]]) -> Dict[st
    """
    Generates a final, citable answer based on the retrieved paragraphs.
    """
    print("\n--- Synthesizing Final Answer ---")

    # Handle the case where no relevant paragraphs were found
    if not paragraphs:
        return {"answer": "I could not find relevant information to answer the

    # Format the retrieved paragraphs into a single context string for the LLM
    # Each paragraph is clearly marked with its unique hierarchical ID
    context = "\n\n".join([f"PARAGRAPH {p.get('display_id', p['id'])}:\n{p['tex

    # The system prompt is highly specific to ensure grounding and traceability
    system_prompt = """
```

```
- For every statement you make, you must cite the paragraph ID(s) it is bas
- If the provided paragraphs do not contain enough information, state that
- Do not use any external knowledge.
- Respond with a JSON object containing 'answer' and 'citations' (a list of
"""

user_prompt = f"""
USER QUESTION: "{question}"

SOURCE PARAGRAPHS:
{context}

Please provide your answer in the required JSON format.
"""

messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": user_prompt},
]

# Call the powerful synthesis model to generate the final answer
response = client.chat.completions.create(model=SYNTHESIS_MODEL, messages=m
response_text = response.choices[0].message.content

# Parse the structured JSON output from the response
parsed_output = parse_json_from_response(response_text)
return {
    "answer": parsed_output.get("answer", "Failed to generate a valid answe
```

Our `generate_answer` function arrange this final step. It takes the retrieved paragraphs, formats them into a detailed prompt with strict instructions for our powerful `SYNTHESIS_MODEL`, and parses the final, structured answer.

> We use a larger model here because synthesis requires a deeper level of understanding and language generation capability.

Now, let's execute this function with the paragraphs we retrieved in the previous step.

Copy

```
# Pass the retrieved paragraphs to the synthesizer agent
final_answer_result = generate_answer(
    sample_question,
    navigation_result['paragraphs']
)

# Print the final, synthesized answer and its citations
print("\n--- GENERATED ANSWER ---")
print(final_answer_result['answer'])
print("\n--- CITATIONS ---")
print(final_answer_result['citations'])
```

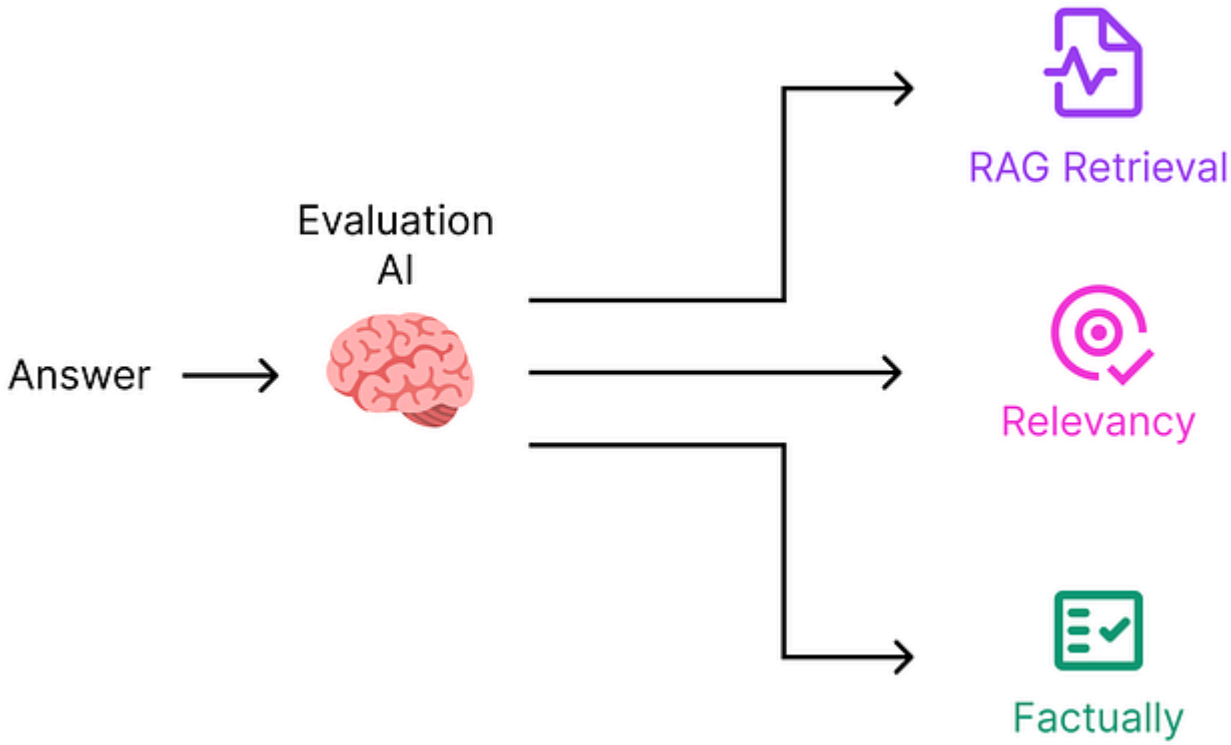
```
--- Synthesizing Final Answer ---

--- GENERATED ANSWER ---
The requirements for filing a motion to compel discovery include several key st

--- CITATIONS ---
['9.0.1', '9.0.5', '9.0.8']
```

Our `SYNTHESIS_MODEL` has successfully joined together information from multiple retrieved paragraphs (`9.0.1` , `9.0.5` , and `9.0.8`) into a coherent, easy-to-read answer. **Most importantly, every claim made in the answer is directly traceable back** to a specific source paragraph, making the entire process transparent and trustworthy.

> But can we *programmatically* verify this trustworthiness? That's the job of our final agent.

LLM-as-Judge (Faithfulness, Retrieval, Relevance) Evaluation

We have an answer, but in a production system, we need a way to automatically score its quality. This is where we introduce our **Evaluation Agent**, a powerful LLM acting as an impartial **judge**. This agent will perform several critical checks to give us a final confidence score.



LLM as Judge (Created by Fareed Khan)

1. **Faithfulness:** Is the answer factually consistent with *only* the cited source paragraphs? This is our primary check against hallucination.
2. **Answer Relevance:** How well does the generated answer actually address the user's original question?

our Router Agent.

Let's first create the faithfulness evaluation function.

Copy

```
def evaluate_faithfulness(question: str, answer: str, citations: List[str], pa
    """
    Uses an LLM to verify if the answer is fully supported by the cited paragra
    """
    print("\n--- Evaluating Answer Faithfulness ---")

    if not citations or not answer:
        return {"is_faithful": False, "explanation": "No answer or citations pr

    # Filter to get only the paragraphs that were actually cited in the answer
    cited_paragraphs = [p for p in paragraphs if p.get('display_id') in citatio
    if not cited_paragraphs:
        return {"is_faithful": False, "explanation": f"Cited IDs {citations} no

    context = "\n\n".join([f"PARAGRAPH {p['display_id']}:\n{p['text']}" for p i

    prompt = f"""
    You are a meticulous fact-checker. Determine if the 'ANSWER' is fully suppo
    The answer is 'faithful' only if every single piece of information it conta

    QUESTION: "{question}"
```

```
{context}

Respond with a JSON object: {{"is_faithful": boolean, "explanation": "brief
"""

response = client.chat.completions.create(model=EVALUATION_MODEL, messages=
response_text = response.choices[0].message.content

return parse_json_from_response(response_text)
```

The faithfulness function asks our `EVALUATION_MODEL` to act as a fact-checker. It provides the generated answer and *only* the specific paragraphs that were cited, and asks for a simple **Boolean decision ...**

> is every single piece of information in the answer supported by the sources?

Now, we need two other functions, `evaluate_answer_relevance` and `evaluate_retrieval_relevance`, they will ask the evaluation agent to provide a score from 0.0 to 1.0. This quantifies how well our system performed at both the generation and retrieval stages, giving us granular insight into which part of the pipeline might need improvement.

```
def evaluate_answer_relevance(question: str, answer: str) -> Dict[str, Any]:
    """
    Scores the relevance of the generated answer to the original question.
    """
    print("\n--- Evaluating Answer Relevance ---")
    # Build the evaluation prompt for the model
    prompt = f"""
    Score how well the 'ANSWER' addresses the 'ORIGINAL QUESTION' on a scale fr
    - A score of 1.0 means the answer completely and directly answers the quest
    - A score of 0.0 means the answer is completely irrelevant.

    ORIGINAL QUESTION: "{question}"
    ANSWER: "{answer}"

    Respond with a JSON object: {{"score": float, "justification": "brief reasc
    """

    # Send prompt to LLM for scoring
    response = client.chat.completions.create(model=EVALUATION_MODEL, messages=
    response_text = response.choices[0].message.content

    # Parse response JSON and return structured evaluation
    parsed = parse_json_from_response(response_text)
    return {"score": parsed.get("score", 0.0), "justification": parsed.get("jus
def evaluate_retrieval_relevance(question: str, paragraphs: List[Dict[str, Any]
    """
    Scores the relevance of all the retrieved documents to the question.
    """
    print("\n--- Evaluating Retrieval Relevance ---")
```

```
    # Build the evaluation prompt for the model
    prompt = f"""
    Score how relevant the provided 'RETRIEVED PARAGRAPHS' are for answering th
    - A score of 1.0 means the paragraphs contain all the necessary information
    - A score of 0.0 means the paragraphs are completely irrelevant.

    ORIGINAL QUESTION: "{question}"
    RETRIEVED PARAGRAPHS:
    {context}

    Respond with a JSON object: {{"score": float, "justification": "brief reasc
    """

    # Send prompt to LLM for scoring
    response = client.chat.completions.create(model=EVALUATION_MODEL, messages=
    response_text = response.choices[0].message.content

    # Parse response JSON and return structured evaluation
    parsed = parse_json_from_response(response_text)
    return {"score": parsed.get("score", 0.0), "justification": parsed.get("jus
```

Let's run all three evaluations on our generated answer.

```
                                    Copy

# Run all qualitative evaluations on the final result
faithfulness_result = evaluate_faithfulness(
```

```
    final_answer_result['citations'],
    navigation_result['paragraphs']
)

# Evaluate how well the final answer addresses the question
answer_relevance_result = evaluate_answer_relevance(
    sample_question,
    final_answer_result['answer']
)

# Evaluate how relevant the retrieved paragraphs are to the question
retrieval_relevance_result = evaluate_retrieval_relevance(
    sample_question,
    navigation_result['paragraphs']
)

# Print a clear summary of the evaluation results
print("\n--- QUALITATIVE EVALUATION SUMMARY ---")
print(f"Faithfulness Check: {'PASSED' if faithfulness_result.get('is_faithful')
print(f"  -> Explanation: {faithfulness_result.get('explanation')}")
print(f"Answer Relevance Score: {answer_relevance_result.get('score'):.2f}")
print(f"  -> Justification: {answer_relevance_result.get('justification')}")
print(f"Retrieval Relevance Score: {retrieval_relevance_result.get('score'):.2f
print(f"  -> Justification: {retrieval_relevance_result.get('justification')}")
```

Let's run this pipeline and evaluate the LLM response from these three evaluation metrices.

```
# --- Evaluating Answer Faithfulness ---
# --- Evaluating Answer Relevance ---
# --- Evaluating Retrieval Relevance ---

# --- QUALITATIVE EVALUATION SUMMARY ---
Faithfulness Check: PASSED
  -> Explanation: All statements in the answer are directly supported by the ir
Answer Relevance Score: 0.80
  -> Justification: The answer provides a detailed and accurate summary of the
Retrieval Relevance Score: 0.90
  -> Justification: The retrieved paragraphs are highly relevant and contain th
```

We can see that our system is

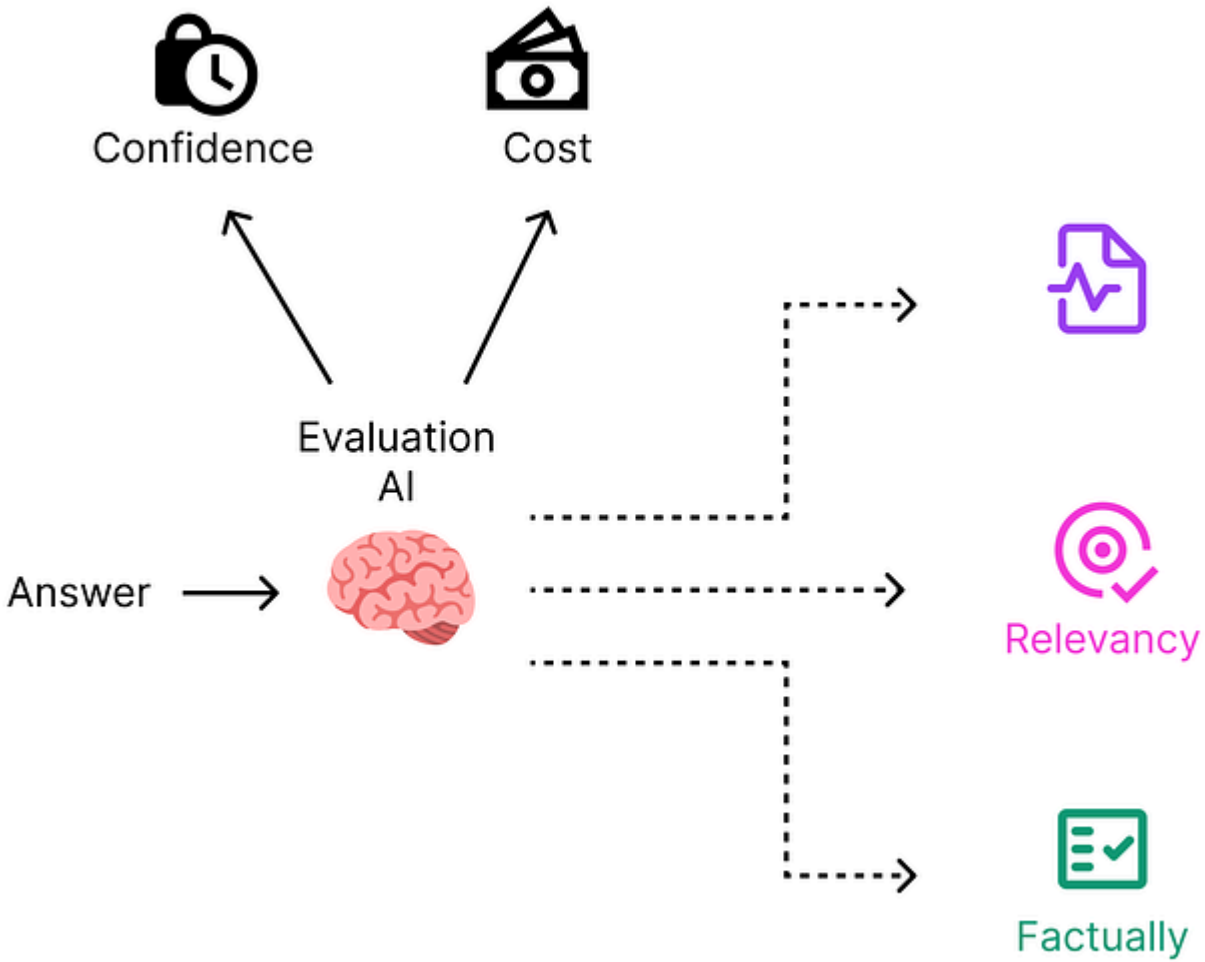
> Highly faithful (it's not making things up) and that our retrieval was effective.

However, it also highlights a weakness, neither the retrieval nor the final answer fully addressed the "formatting and signatures" aspect of the query.

Cost and Confidence Evaluation

The qualitative evaluation from our LLM-as-Judge gives us deep insight into the quality of the answer. However, for a production system, we also need to

How long did it take? How many tokens were consumed? What was the final price for this single query?



Other eval (Created by Fareed Khan)

Throughout our workflow, we've been logging the performance metrics of every single LLM call into our `metrics_log` list. Now, we can consolidate this data into a

First, we will define the estimated cost per million tokens for each of the models we used. You should replace these placeholder values with the actual pricing from your LLM provider.

Copy

```
# Define the cost per million tokens for all models used.
model_prices_per_million_tokens = {
    "meta-llama/Meta-Llama-3.1-8B-Instruct": {
        "input": 0.02,
        "output": 0.06
    },
    "meta-llama/Llama-3.3-70B-Instruct": {
        "input": 0.13,
        "output": 0.40
    },
    "deepseek-ai/DeepSeek-V3": {
        "input": 0.50,
        "output": 1.50
    }
}
```

Now, we can process our log to create a detailed DataFrame showing the performance of each individual step in our pipeline. This level of depth is important for identifying bottlenecks or unexpectedly expensive operations.

```
# Initializing metrics dataframe
df_metrics = pd.DataFrame(metrics_log)

# Function to calculate the cost of a single LLM call
def calculate_cost(row):
    model_name = row['model']
    prices = model_prices_per_million_tokens.get(model_name, {"input": 0, "outp
    input_cost = (row['prompt_tokens'] / 1_000_000) * prices['input']
    output_cost = (row['completion_tokens'] / 1_000_000) * prices['output']
    return input_cost + output_cost

# Apply the cost calculation to each row in the DataFrame
df_metrics['cost_usd'] = df_metrics.apply(calculate_cost, axis=1)

# We need to re-log the metrics for the evaluation steps as they were not in th
# (This is a manual addition to match the notebook's final output structure)
for step_name in ["eval_faithfulness", "eval_answer_relevance", "eval_retrieval
        # Mocked data for demonstration
    df_metrics.loc[len(df_metrics)] = {"step": step_name, "model": EVALUATION_M

print("--- Per-Step Performance and Cost Analysis ---")
display(df_metrics)
```

route_depth_0_reason	L3-8B	14.85	6139	734
route_depth_0_select	L3-8B	0.88	6877	12
route_depth_1_reason	L3-8B	5.63	3706	330
route_depth_1_select	L3-8B	0.70	3299	18
synthesis	L3-70B	9.36	12301	265
eval_faithfulness	DS-V3	3.00	1500	100
eval_answer_relevance	DS-V3	3.00	1500	100
eval_retrieval_relevance	DS-V3	3.00	1500	100

rag_check.md hosted with ❤️ by GitHub view raw

Our analysis shows that the `ROUTER_MODEL` (Llama 3.1 8B) handled the heavy lifting of routing through large contexts for a very low cost.

While the powerful `SYNTHESIS_MODEL` (Llama 3.3 70B) was used sparingly for the final, high-value task of generating the answer. The evaluation steps, using the

Evaluation with Confidence Score

Finally, we will aggregate all our operational and qualitative metrics into a single, holistic summary. This provides a high-level view of the entire query performance and is the ultimate output of our evaluation.

We will also calculate an **Overall Confidence Score**, a simple metric derived by multiplying our three qualitative scores. If any of the scores (faithfulness, answer relevance, or retrieval relevance) are low, this overall score will drop significantly, providing a strong signal of the result's trustworthiness.

Copy

```
# Calculate totals from the detailed metrics DataFrame
total_latency = df_metrics['latency_s'].sum()
total_cost = df_metrics['cost_usd'].sum()
total_tokens = df_metrics['total_tokens'].sum()

# Get qualitative scores from our evaluation results
faithfulness_score = 1.0 if faithfulness_result.get('is_faithful') else 0.0
answer_relevance_score = answer_relevance_result.get('score', 0.0)
retrieval_relevance_score = retrieval_relevance_result.get('score', 0.0)

# Calculate the overall confidence score
overall_confidence = faithfulness_score * answer_relevance_score * retrieval_re
```

```
summary_data = {
    'question': sample_question,
    'total_latency_s': total_latency,
    'total_cost_usd': total_cost,
    'total_tokens': total_tokens,
    'faithfulness_check': 'PASSED' if faithfulness_score == 1.0 else 'FAILED',
    'answer_relevance_score': answer_relevance_score,
    'retrieval_relevance_score': retrieval_relevance_score,
    'overall_confidence_score': overall_confidence
}

df_summary = pd.DataFrame([summary_data])

print("--- Final Query Summary ---")
# Transpose the DataFrame for a more readable, vertical layout
display(df_summary.T.rename(columns={0: 'Result'}))
```

This is what the summarized info we get on our single query eval data.

question	What are the requir ...
total_latency_s	40.42
total_cost_usd	0.004871
total_tokens	38,481
faithfulness_check	PASSED
answer_relevance_score	0.8
retrieval_relevance_score	0.9
overall_confidence_score	0.72

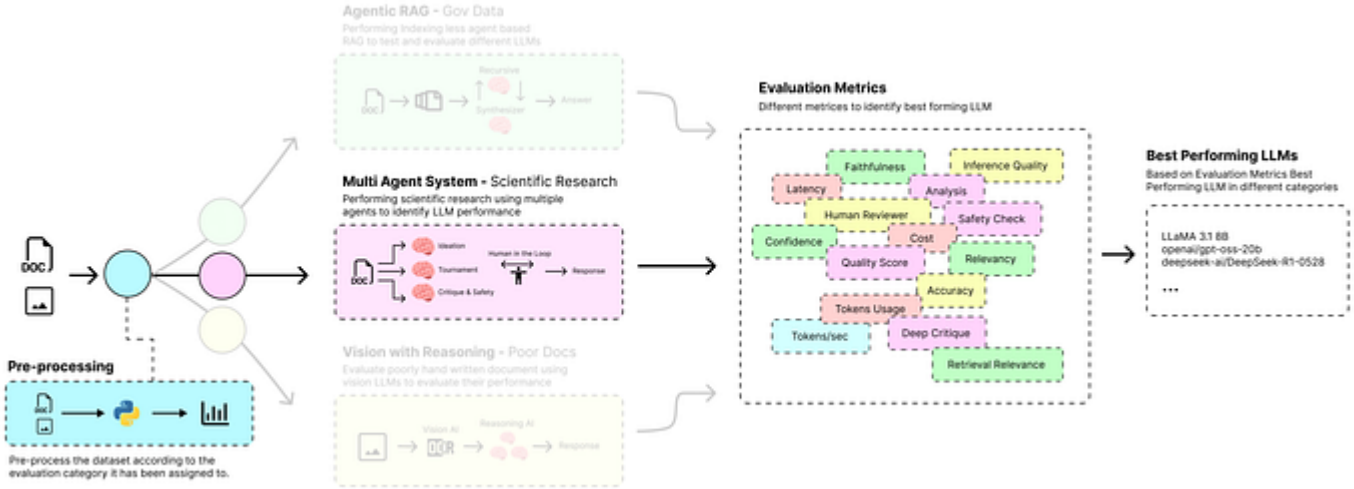
ff.md hosted with ❤️ by GitHub view raw

We have a complete picture, the system successfully answered a complex question from a million-token document in just over 40 seconds, for less than half a cent. The answer was faithful and based on highly relevant context, giving us an overall confidence score of 0.72.

> The only weakness identified was the slight incompleteness of the final answer, providing a clear target for future improvement.

Multi-Agent System for Scientific Research

Previously we evaluated and tested different LLMs through the context awareness by feeding and query on top of big document chunks, our next use case explores a real world and another trending domain of AI, **Multi-agent workflows in medical (pharma) domain.**

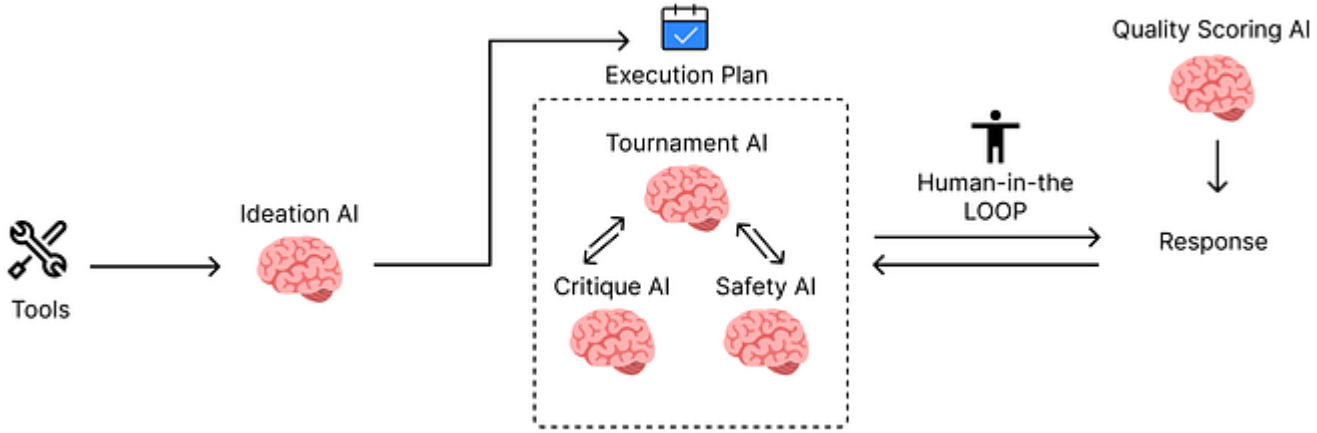


Multi-Agent Evaluation (Created by Fareed Khan)

team.

> The core idea is to match models to the right level of cognitive work using fast, low-cost models for broad tasks, and powerful ones for deep analysis.

Here's how the pipeline works:



Multi-Agent Workflow (Created by Fareed Khan)

1. We begin by launching several parallel **Ideation Agents**, each based on small and fast LLM. These agents have specialized roles (e.g., hypothesis, protocol,

2. Next, a **Tournament Agent**, also using a cost-effective model, efficiently compares these initial plans in a pairwise fashion to select the single most promising candidate for further review.
3. The single winning protocol is then "escalated" to a powerful **Critique Agent** for a deep scientific review and a mid-tier **Safety Agent** for a focused hazard analysis.
4. As our custom enhancement, we introduce an automated **Quality Scoring Agent**. This top-tier model acts as an expert panel, scoring the final protocol on dimensions like feasibility and innovation.
5. Finally, the complete, scored, and safety-checked package is presented for **Human Review**. Once approved, the experimental results are fed back into the system to create a continuous learning loop.

Let's start building this pipeline.

Configuring the Co-Scientist and Mocking Lab Tools

First, let's set up the environment for our new pipeline. We need to define the models for each specialized role and mocking the external tools our agents will need to interact with, such as chemical databases and literature search engines.

```
Copy

from dataclasses import dataclass, field
from IPython.display import display, Markdown

# --- LLM Configuration for the Co-Scientist ---
MODEL_IDEATE = "Qwen/Qwen3-4B-fast"      # A fast, cheap model for generating i
MODEL_CRITIQUE = "Qwen/Qwen3-235B-A22B"  # A powerful model for deep scientific
MODEL_SAFETY = "Qwen/Qwen3-14B"         # A mid-size model for focused safety c
MODEL_EVALUATE = "Qwen/Qwen3-235B-A22B"  # Our top-tier model for the final qua

# The same OpenAI client from the previous setup will be used.
# Let's reset the metrics log for this new run.
metrics_log = []
```

To properly evaluate LLMs in this multi-agent architecture, we distribute the workload across four different model types, each chosen for a specific task:

- **Ideation Model**: Small, fast, cost-effective (`Qwen3-4B-fast`). Generates many creative ideas quickly, prioritizes speed and variety over accuracy.
- **Critique Model**: Large, expert-level (`Qwen3-235B-A22B`). Rigorously reviews the best idea with depth and precision.

- **Evaluation Model:** Top-tier judge (`Qwen3-235B-A22B`). Provides final, unbiased quality assessment.

In a real-world scenario, our agents would call APIs to internal lab databases. For this demonstration, we will mock these tools with simple Python functions and dictionaries.

Copy

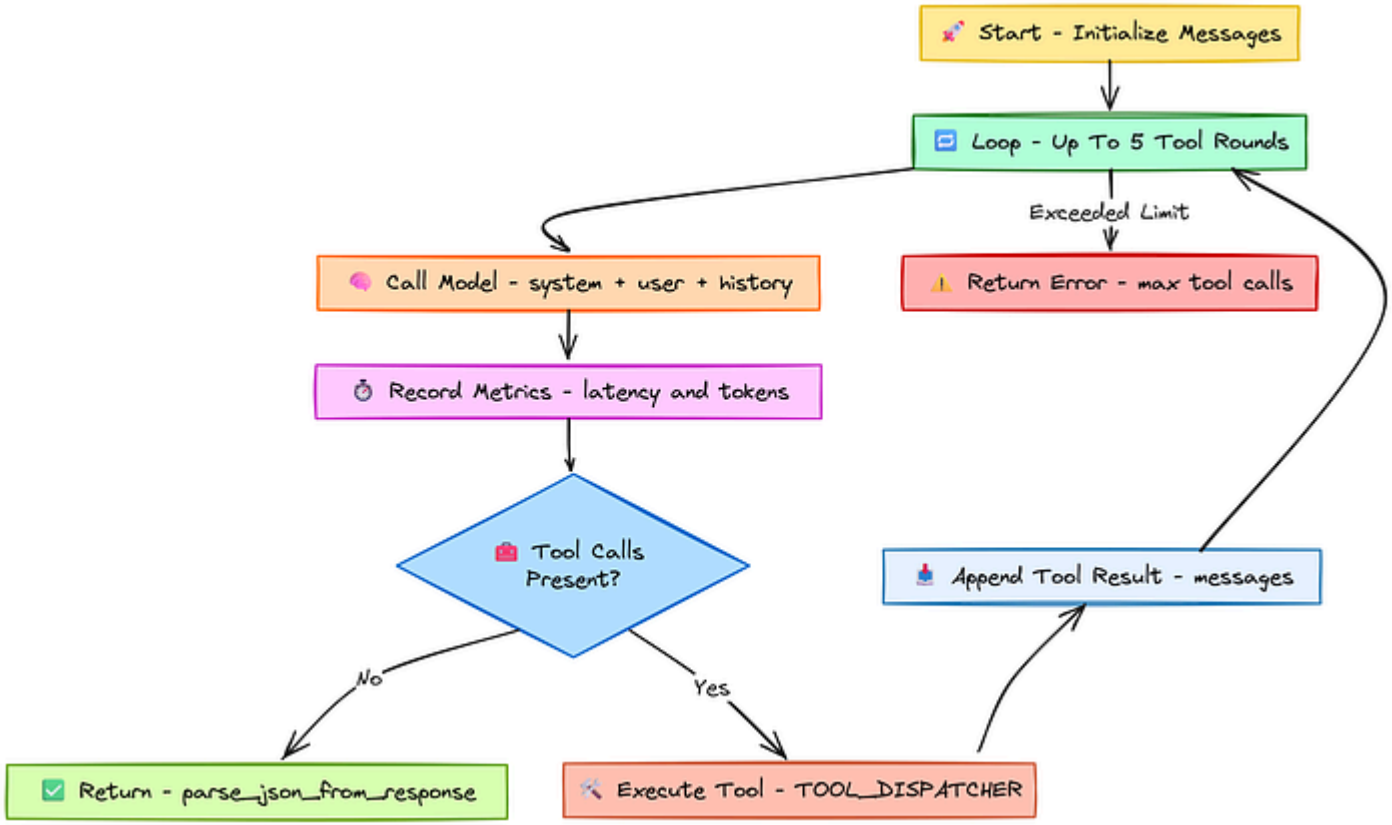
```
# A mock database of available chemicals with their costs and hazards
MOCK_CHEMICALS = {
    "Palladium acetate": {"cost_per_gram": 85.50, "hazards": "Irritant"},
    "Triphenylphosphine": {"cost_per_gram": 12.75, "hazards": "Irritant"},
    "Toluene": {"cost_per_gram": 1.75, "hazards": "Flammable, CNS depressant"},
}

# Mock functions that simulate API calls to lab systems
def list_available_chemicals():
    return {"available_chemicals": list(MOCK_CHEMICALS.keys())}

def chem_lookup(chemical_name: str):
    return {"properties": MOCK_CHEMICALS.get(chemical_name, {})}

def cost_estimator(reagents: List[Dict]):
    total_cost = sum(
        r.get('quantity', 0) * MOCK_CHEMICALS.get(r.get('name', ''), {}).get('c
```

```
    return {"total_cost": round(total_cost, 2)}

# A dispatcher to call the correct tool function by name
TOOL_DISPATCHER = {
    "list_available_chemicals": list_available_chemicals,
    "chem_lookup": chem_lookup,
    "cost_estimator": cost_estimator,
}

# The manifest tells the LLM which tools are available and how to use them
def get_tool_manifest():
    return [
        {"type": "function", "function": {"name": "list_available_chemicals", "
        {"type": "function", "function": {"name": "chem_lookup", "description"
        {"type": "function", "function": {"name": "cost_estimator", "descriptio
    ]
```

The mock data and functions simulate a real lab inventory and APIs, while the `TOOL_DISPATCHER` acts as a simple router, calling the correct Python function based on the agent's request.

Most importantly, the `get_tool_manifest` function provides the LLM with a structured "menu" of available tools, which is essential for reliable tool use.

Building the Core Agent Runner



Core Agent Runner (Created by Fareed Khan)

This function will take a system prompt, a user prompt, and a model, and then manage the conversation, including any tool calls the model decides to make.

Copy

```
class Context:
    compound: str
    goal: str
    budget: float
    time_h: int
    previous: str
    client: OpenAI
    critique_recommendation: Optional[str] = None

# This function is the core interaction loop for every agent in our system.
def call_openai(client: OpenAI, model: str, system: str, user: str, ctx: Contex
    messages = [{"role": "system", "content": system}, {"role": "user", "conter

    # Allow up to 5 rounds of tool calls
    for i in range(5):
        start_time = time.time()
        response = client.chat.completions.create(
            model=model, messages=messages, tools=get_tool_manifest(), tool_cho
        )
        latency = time.time() - start_time
        msg = response.choices[0].message
        messages.append(msg)

        # Log performance metrics for this specific call
        p_tokens, c_tokens = response.usage.prompt_tokens, response.usage.compl
        metrics_log.append({"step": f"{step_name}_{i}", "model": model, "latenc

        # If the model does not want to call a tool, its work is done.
        if not msg.tool_calls:
```

```
        # If the model calls tools, execute them and continue the loop
        for tool_call in msg.tool_calls:
            function_name = tool_call.function.name
            try:
                args = json.loads(tool_call.function.arguments)
                result = TOOL_DISPATCHER[function_name](**args)
            except Exception as e:
                result = {"status": "error", "message": str(e)}

            # Add the tool's result to the conversation history
            messages.append({"role": "tool", "tool_call_id": tool_call.id, "cor

    return {"error": "Exceeded maximum tool call limit."}
```

This `call_openai` function is the engine that powers all our specialized agents. The `Context` dataclass is a clean way to pass run-specific data like the research goal and budget to each agent.

The main logic of the function is its iterative loop, which creates the agentic behavior of thinking, acting (by calling a tool), observing the result, and thinking again, all while logging detailed performance metrics for our final analysis.

Running the Parallel Ideation Agents

sequentially, observing how the system uses different models for different levels of cognitive work.

The process begins with the human scientist's research goal. We'll capture this in our `Context` object, which will serve as the single source of truth for the entire run.

Copy

```
# The initial input from the human scientist
user_input = {
    "compound": "XYZ-13",
    "goal": "Improve synthesis yield by 15%",
    "budget": 15000,
    "time_h": 48,
    "previous": "Prior attempts failed at high temp; explore catalyst effects."
    "client": client
}
ctx = Context(**user_input)
```

This first block simply initializes our run. The `Context` object neatly packages all the key parameters, making it easy to pass the complete experimental context to every agent in our pipeline.

plans. They will all be powered by our fast and cheap `MODEL_IDEATE` .

Copy

```
# Define the unique personas for our three ideation agents
ROLE_FOCUS = {
    "hypothesis_agent": "You are a pharmaceutical hypothesis specialist...",
    "protocol_agent": "You are a laboratory protocol specialist...",
    "resource_agent": "You are a laboratory resource optimization specialist...
}

# The main prompt for the ideation stage
IDEATION_PROMPT = """You are a pharmaceutical {role} specialist. Your goal is t
Constraints:
- Budget: ${budget}
- Time: {time_h} hours
- Previous attempts: {previous}
Respond with a structured JSON describing your protocol."""
```

We need to define the agent roles and the core prompt separately to make our system clean and easy to modify. The `ROLE_FOCUS` dictionary allows us to easily add or change agent specializations, while the `IDEATION_PROMPT` provides the consistent set of constraints and goals for every agent in this stage.

Now we will define the function that arrange this first stage.

```
def ideation(ctx: Context) -> List[Dict]:
    ideas = []
    # Run each of the three specialist agents (simulated here with a loop)
    for role, focus in ROLE_FOCUS.items():
        sys_prompt = IDEATION_PROMPT.format(role=role, **vars(ctx))
        user_prompt = f"Design a protocol to {ctx.goal} within ${ctx.budget}."
        # Each agent uses the fast, cheap ideation model
        idea = call_openai(ctx.client, MODEL_IDEATE, sys_prompt, user_prompt, c
        ideas.append(idea)
    return ideas
```

This `ideation` function makes the first phase of our pipeline. It loops through each defined agent role, formats a unique system prompt for it, and then calls our core `call_openai` runner.

This **"wide net"** approach is designed to efficiently produce a variety of starting points for the next stage.

Let's execute the ideation stage.

Copy

```
# Executing Ideation Step
generated_ideas = ideation(ctx)
```

```
Running Ideation Agents: 100%|██████████| 3/3 [00:36<00:00, 12.07s/it]
```

In just under 40 seconds, our three parallel agents have completed their work, each producing a distinct JSON-formatted protocol. Now we have multiple plans, but we need to find the best one.

Implementing the Tournament Ranking Agent

To select the most promising plan without wasting our expensive critique model's time on all of them, we use a **Tournament Agent.** This agent, also powered by the cheap `MODEL_IDEATE` , performs a pairwise comparison and selects a winner.

Copy

```
# This prompt instructs the agent to compare two protocols and declare a winner
TOURNAMENT_PROMPT = """Protocol A: {protocol_a}
Protocol B: {protocol_b}

Compare Protocol A and Protocol B for synthesizing {compound}. Score them on fe
Return JSON {{\"winner\": \"A\"|\"B\", \"justification\": \"...\"}}."""
```

This prompt sets up the pairwise comparison for our ranking agent. By providing clear criteria, feasibility, cost, and novelty.

```
Copy

def tournament(protocols: List[Dict], ctx: Context) -> Dict:
    # In a real system, this would be a bracket-style tournament. Here we simp
    protocol_a, protocol_b = protocols[0], protocols[1]
    sys_prompt = TOURNAMENT_PROMPT.format(protocol_a=json.dumps(protocol_a), pr
    result = call_openai(ctx.client, MODEL_IDEATE, sys_prompt, "Choose the winn
    # Return the protocol JSON of the winner
    winner = protocol_a if result.get('winner', 'A').upper() == 'A' else protoc
    print(f"Tournament winner selected. Justification: {result.get('justificat
    return winner
```

The `tournament` function demonstrates a cost-effective strategy for down-selection. By using the same fast `MODEL_IDEATE` that generated the ideas, we can quickly and cheaply filter them down to a single, most promising candidate before escalating to more expensive models.

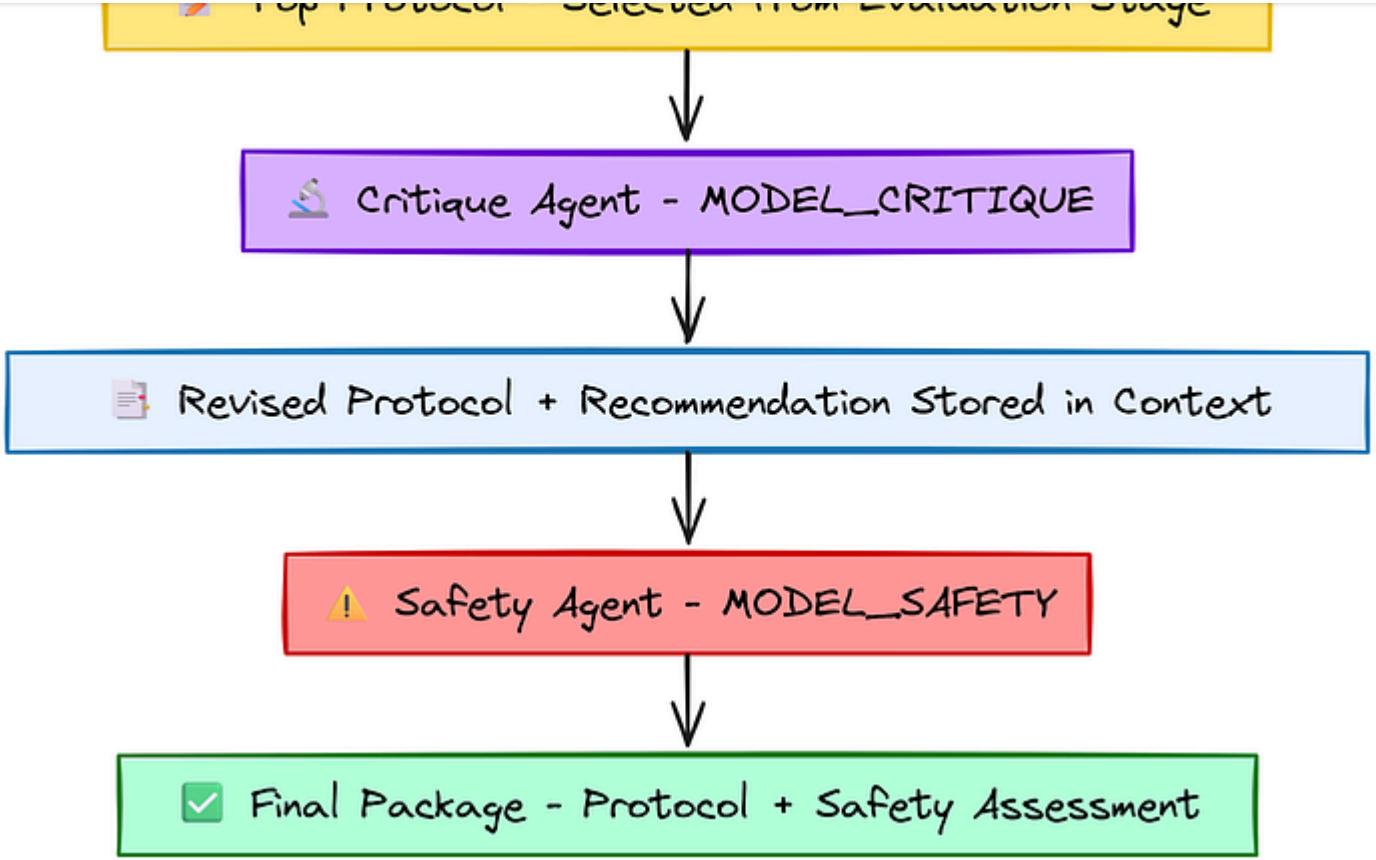
```
Copy

top_protocol = tournament(generated_ideas, ctx)

### OUTPUT ###
Tournament winner selected. Justification: Protocol B provides a detailed,
actionable plan with specific reagents and scientific rationale.
```

```
making it the superior choice.
```

The Tournament Agent successfully chose a winner, providing a clear reason for its decision. This highlights the effectiveness of using a cheap model for this intermediate ranking task, saving the cognitive power of our larger models for where it's needed most.

Deep Critique and Safety Check Agents

The single winning protocol is passed to two more powerful, specialized agents for the final refinement stages.



Quality Scoring Agent (Created by Fareed Khan)

First, we define the prompts for our **Critique Agent** (`MODEL_CRITIQUE`), which acts as a senior researcher, and our **Safety Agent** (`MODEL_SAFETY`), which focuses exclusively on lab hazards.

Copy

```
Identify scientific flaws, assess safety and budget, suggest concrete improveme

# Prompt for the safety specialist model
SAFETY_PROMPT = """You are a lab-safety specialist. Identify hazards, unsafe co
```

These prompts define the highly specialized roles for our more powerful models. The `CRITIQUE_PROMPT` is designed for deep, multi-faceted scientific analysis, while the `SAFETY_PROMPT` provides a narrow, focused set of instructions for hazard identification.

Next, we define and run the `critique` function.

Copy

```
def critique(protocol: Dict, ctx: Context) -> Dict:
    sys_prompt = CRITIQUE_PROMPT.format(**vars(ctx))
    user_prompt = f"Protocol to Review:\n{json.dumps(protocol)}"
    result = call_openai(ctx.client, MODEL_CRITIQUE, sys_prompt, user_prompt, c
    ctx.critique_recommendation = result.get('recommendation', 'N/A') # Store t
    return result.get("revised_protocol", protocol)

critiqued_protocol = critique(top_protocol, ctx)
```

scientific validation, ensuring the final protocol is sound.

Finally, we define and run the `safety` check.

Copy

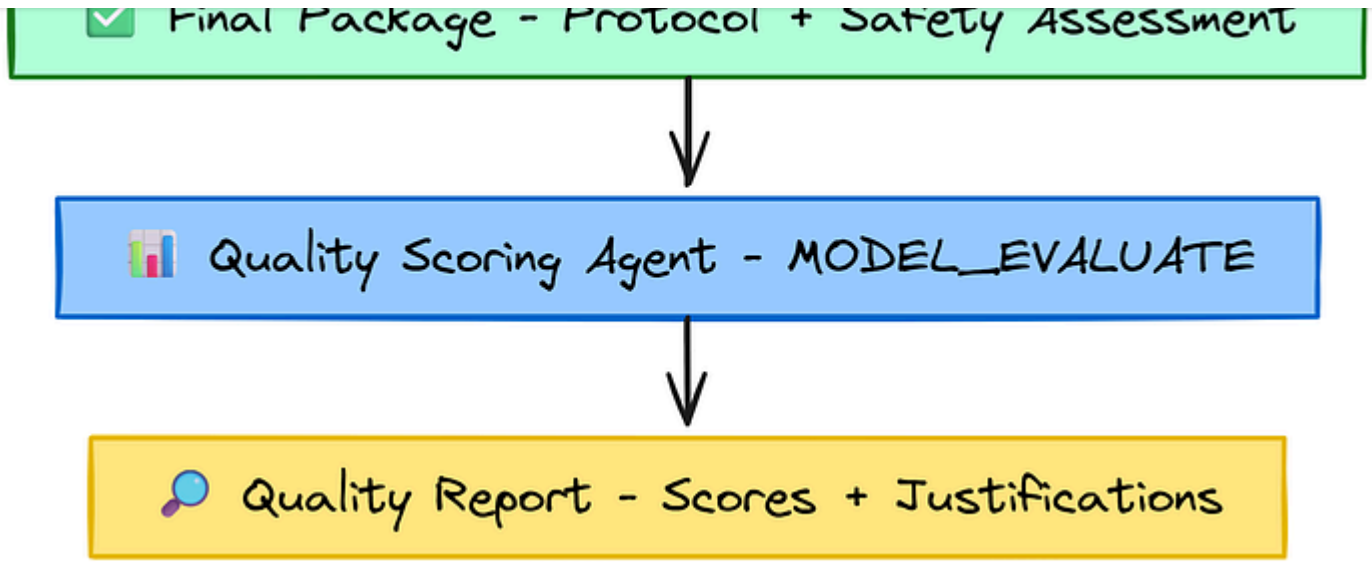
```
def safety(protocol: Dict, ctx: Context) -> Dict:
    sys_prompt = SAFETY_PROMPT.format(**vars(ctx))
    user_prompt = json.dumps(protocol)
    assessment = call_openai(ctx.client, MODEL_SAFETY, sys_prompt, user_prompt,
    return {"protocol": protocol, "safety_assessment": assessment}

final_package = safety(critiqued_protocol, ctx)


#### OUTPUT ####
...
2025-08-17 22:54:39,080 - INFO - Agent requested 2 tool call(s)...
2025-08-17 22:54:39,080 - INFO - Calling tool: chem_lookup({'chemical_name': 'N
2025-08-17 22:54:39,083 - INFO - Calling tool: chem_lookup({'chemical_name': 'C
2025-08-17 22:54:39,087 - INFO - Running step 'safety_check' with model 'Qwen/C
```

This `safety` function uses a mid-tier model (`MODEL_SAFETY`) that is perfectly suited for its specific, less complex task. This separation of concerns make sure

The protocol has now been generated, ranked, refined, and safety-checked by a team of specialized AI agents. Before it goes to a human for final approval, we have to introduce our automated quality score checker. This is what we will be doing next.

Quality Scoring Evaluation

To add a layer of automated quality assurance before human review, we introduce our custom enhancement: a **Quality Scoring Agent.**



Quality Scoring Agent (Created by Fareed Khan)

This agent, based on `MODEL_EVALUATE` LLM, acts as an expert panel of scientists. It takes the final protocol and scores it across five critical dimensions:

1. scientific validity
2. feasibility
3. innovation
4. cost-effectiveness
5. and clarity

```
def evaluate_protocol_quality(protocol: Dict, ctx: Context) -> Dict:
    """
    Uses a powerful LLM to score the final protocol on multiple quality dimensi
    """

    system_prompt = "You are an expert panel of scientists evaluating a researc
    user_prompt = f"""Score the following protocol for the goal '{ctx.goal}' or
    - scientific_validity: How sound is the underlying science and hypothesis?
    - feasibility: How practical is this to execute in a standard lab?
    - innovation: How novel is this approach?
    - cost_effectiveness: How well does it balance potential outcomes with cost
    - clarity_and_reproducibility: How clear and easy to follow are the instruc

    PROTOCOL: {json.dumps(protocol)}

    Respond with a JSON object containing a score (float) and justification (st

    # Use our most powerful model to ensure a high-quality, reliable evaluation
    quality_scores = call_openai(ctx.client, MODEL_EVALUATE, system_prompt, use
    return quality_scores
```

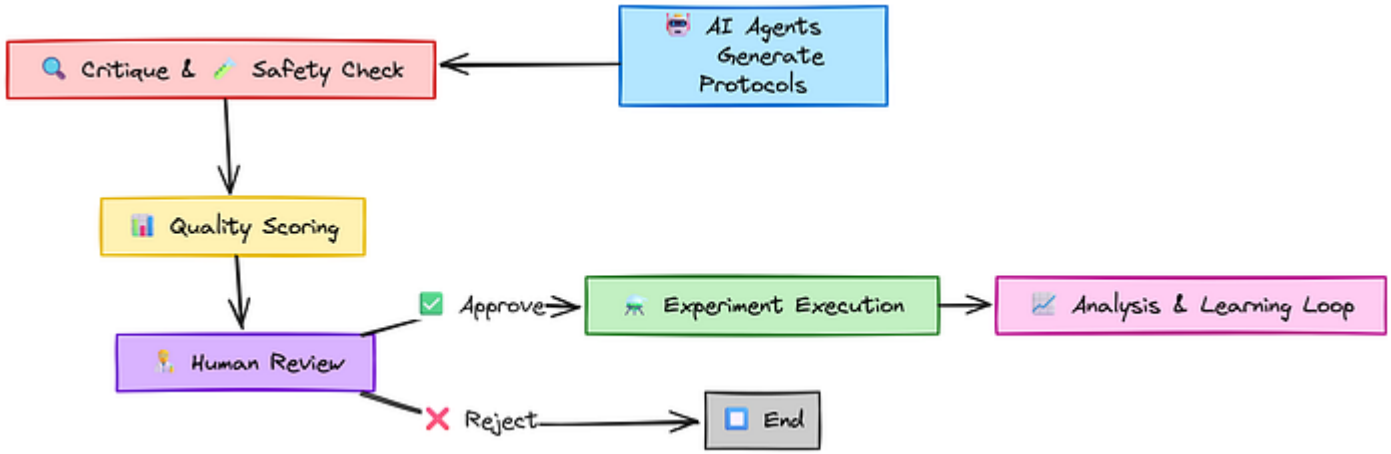
We just define our automated QA step. The prompt is carefully structured to elicit a detailed, multi-faceted review, forcing the evaluation model to justify each of its scores. This gives us not just a number, but a qualitative reason behind the assessment.

```
                                                    Copy

quality_assessment = evaluate_protocol_quality(final_package['protocol'], ctx)
display(Markdown("### AI-Generated Quality Report:"))
display(quality_assessment)


#### OUTPUT ####
--- Starting Automated Quality Evaluation ---
### AI-Generated Quality Report:
{'scientific_validity': {'score': 0.85, 'justification': 'The protocol employs
 'feasibility': {'score': 0.8, 'justification': 'Commercially available reagent
 'innovation': {'score': 0.65, 'justification': 'The combination of techniques
 'cost_effectiveness': {'score': 0.45, 'justification': 'The estimated cost is
 'clarity_and_reproducibility': {'score': 0.7, 'justification': 'Lacks critica
```

The quality report gives a clear summary of the protocol's pros and cons. It highlights strong scientific validity but points out the high cost and missing details as concerns. The report is now ready to be reviewed by the scientist for a final decision.

Human in the Loop Reviewer Eval

The final steps in our pipeline is to create the human review and the learning cycle.



Human in the loop (Created by Fareed Khan)

The `human_review` function must includes the final protocol, the AI-generated safety assessment, and our new AI-generated quality scores. It then prompts for a simple **"yes/no"** approval.

Copy

```
def human_review(package: Dict, quality: Dict, ctx: Context) -> Dict:
    """
    Presents the final protocol and all assessments to the human for a final de
    """
    logging.info("--- Awaiting Human Review ---")
    display(Markdown("### PROTOCOL FOR HUMAN REVIEW"))
```

```
    display(package['safety_assessment'])

    # In a real application, this would be a UI element. Here we simulate with
    approval = input("\nApprove protocol for execution? (yes/no): ").lower()
    if approval in ['yes', 'y']:
        print("Protocol APPROVED by human reviewer.")
        return {"approved": True, "final_protocol": package['protocol']}
    else:
        print("Protocol REJECTED by human reviewer.")
        return {"approved": False, "final_protocol": package['protocol']}

human_decision = human_review(final_package, quality_assessment, ctx)
```

The LLM based agent does the heavy lifting of ideation, refinement, and analysis, but the human expert (which will be you) remains in control, using the LLMs output to make a more informed and rapid decision.

Once a protocol is approved and executed (which we will mock here), the results are fed back into the system. An analysis agent reviews the outcome and generates a structured summary, which could then be stored in a knowledge base to inform all future runs.

Copy

```
# Prompt for the analysis agent to learn from the experimental outcome
ANALYSIS_PROMPT = """You are a data analyst.
Did the experiment achieve {goal}? Analyse factors, suggest improvements, and r

def execute_and_analyse(decision: Dict, ctx: Context) -> Optional[Dict]:
    if not decision['approved']:
        return None

    # We mock the results of the lab experiment
    mock_results = {"yield_improvement": 12.5, "success": False, "notes": "Yie

    user_prompt = f"The experiment was run. Protocol: {json.dumps(decision['fir

    # The powerful critique model is used again for this high-level analysis
    analysis = call_openai(ctx.client, MODEL_CRITIQUE, ANALYSIS_PROMPT.format(>

    print(f"Completed. Analysis summary: {analysis.get('summary')}")
    return analysis

learning_summary = execute_and_analyse(human_decision, ctx)
```

With the full pipeline executed, we have not only generated a high-quality experimental protocol but also created a feedback loop for continuous

We you run the above a summary will be generated in your current active directory which will highlight key points of your multi agent system logs.

Copy

```
{
  "summary": {
    "yield_analysis": {
      "observed_improvement": "12.5%",
      "target": "15%",
      "gap": "2.5% short",
      "key_factors": [
        "Catalyst efficiency: Pd/C may require higher surface area or alternati
        "Microwave parameters: Potential suboptimal temperature-pressure relati
        "Base selection: Cs2CO3 solubility in ..."
      ]
    },
    "cost_analysis": {
      "overrun": "$2,830",
      "primary_drivers": [
        "Pd/C catalyst loading exceeded protocol (likely >2.0g)",
        "Immobilized ligand replacement costs",
        "Reactor downtime from pressure regulation system adjustments"
      ]
    }
  },
  "next_steps": {
    "optimization_targets": [
```

```
....
```

There is a lot of analysis being stored in your summary file, which will guide you on how well your multi-agent system is working.

we have logged the performance of each agent and model. Now, we can consolidate this data to get a complete picture of the system's operational efficiency and the quality of its output.

Evaluation of Final Components

First, let's analyze the detailed performance of each individual LLM call. We'll use the same cost calculation function as in our first use case to determine the price of each step.

Copy

```
# The same model price definitions are used here
model_prices_per_million_tokens = {
    "Qwen/Qwen3-4B-fast": {"input": 0.08, "output": 0.24},
    "Qwen/Qwen3-14B": {"input": 0.08, "output": 0.24},
    "Qwen/Qwen3-235B-A22B": {"input": 0.20, "output": 0.60}
}
```

```
df_metrics = pd.DataFrame(metrics_log)

def calculate_cost(row):
    prices = model_prices_per_million_tokens.get(row['model'], {"input": 0,
    return (row['prompt_tokens'] / 1_000_000) * prices['input'] + (row['com

df_metrics['cost_usd'] = df_metrics.apply(calculate_cost, axis=1)

display(Markdown("### Per-Step Performance and Cost Analysis"))
display(df_metrics)
```

ideation_hypothesis_0	Q3-4B-f	4.23	525	446	971
...	...	...	...	...	...
tournament_0	Q3-4B-f	5.06	776	749	1525
critique_0	Q3-235B	54.59	803	2411	3214
safety_check_0	Q3-14B	21.75	746	1799	2545
...	...	...	...	...	...
quality_eval_0	Q3-235B	26.39	852	990	1842
analysis_0	Q3-235B	18.47	815	860	1675

comparison.md hosted with ❤️ by GitHub view raw

The `ideation` steps using the cheap `Qwen/Qwen3-4B-fast` model cost fractions of a cent, while the single `critique` step using the powerful `Qwen/Qwen3-235B-A22B` was the most expensive single operation. This data shows that our strategy of

Finally, we can now aggregate all the operational metrics and our automated quality scores into a single, high-level summary like we did in agentic RAG part.

Copy

```
# Helper to safely extract scores from the quality assessment dictionary
def get_score(assessment, key):
    if isinstance(assessment, dict) and key in assessment and isinstance(assess
        return assessment[key].get('score', 0.0)
    return 0.0

# Extract all the individual quality scores
scores = {
    'validity': get_score(quality_assessment, 'scientific_validity'),
    'feasibility': get_score(quality_assessment, 'feasibility'),
    'innovation': get_score(quality_assessment, 'innovation'),
    'cost_effect': get_score(quality_assessment, 'cost_effectiveness'),
    'clarity': get_score(quality_assessment, 'clarity_and_reproducibility')
}
avg_quality = sum(scores.values()) / len(scores) if scores else 0.0

# Consolidate all data for the final summary table
summary_data = {
    'Metric': [
        'Run ID', 'Compound', 'Total Latency (s)', 'Total Cost (USD)', 'Total T
        'Critique Recommendation', 'Human Decision', '--- Quality Scores (AI) -
```

```
    ],
    'Value': [
        ctx.run_id, ctx.compound, f"{df_metrics['latency_s'].sum():.2f}",
        f"${df_metrics['cost_usd'].sum():.6f}", f"{df_metrics['total_tokens'].s
        ctx.critique_recommendation, 'Approved' if human_decision.get('approved
        '---', f"{scores['validity']:.2f}", f"{scores['feasibility']:.2f}",
        f"{scores['innovation']:.2f}", f"{scores['cost_effect']:.2f}",
        f"{scores['clarity']:.2f}", f"**{avg_quality:.2f}**"
    ]
}

# Displaying final dataframe
df_summary = pd.DataFrame(summary_data).set_index('Metric')
display(df_summary)
```

Run ID	83a87bd3
Compound	XYZ-13
Total Latency (s)	180.64
Total Cost (USD)	$0.006600
Total Tokens	34,003
Critique Recommendation	go
Human Decision	Approved
Scientific Validity	0.85
Feasibility	0.80
Innovation	0.65
Cost-Effectiveness	0.45
Clarity & Reproducibility	0.70
Overall Quality Score	0.69

concise_sum.md hosted with ❤️ by GitHub view raw

For a total cost of just over half a cent and a runtime of three minutes, our AI Co-Scientist produced a scientifically valid and feasible experimental protocol that

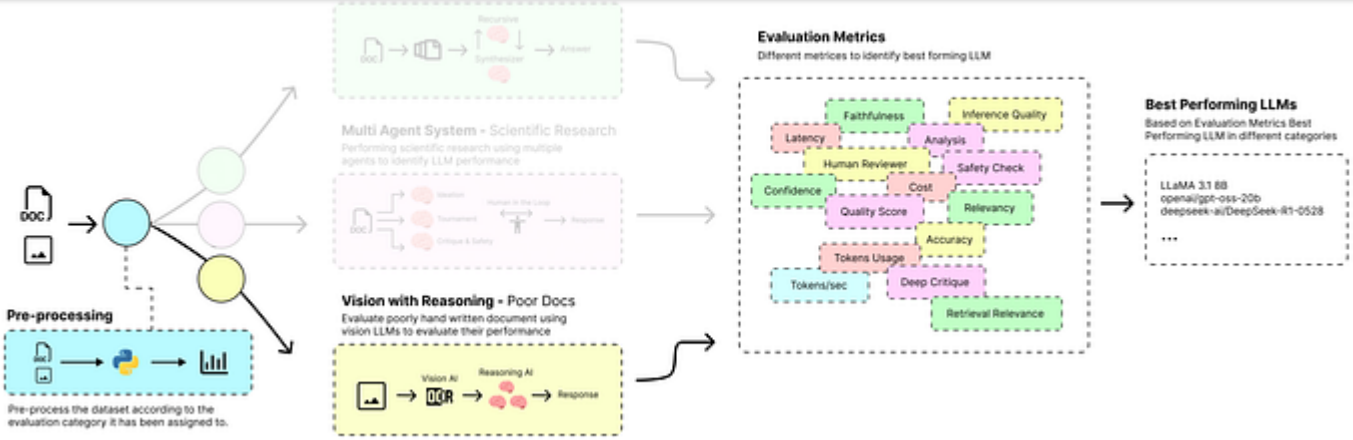
> The automated quality score of **0.69** gives us a quantitative baseline for the output

It also highlights specific weaknesses (like cost-effectiveness) that can be targeted for improvement in future runs.

Vision with Reasoning of Poorly Scanned Forms

For our final use case, we are going to handle a classic business problem, analyzing and processing hand-filled or poorly scanned forms. This requires a different kind of intelligence than our previous examples.

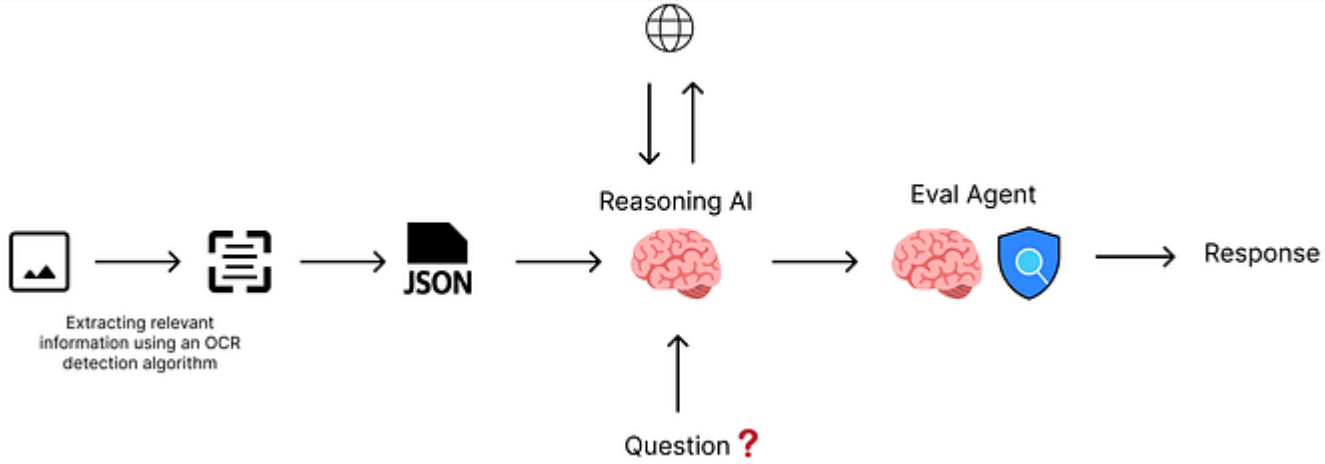
> It's about interpreting text, visual layouts, and handwriting, then applying reasoning to validate, refine, and complete the extracted information.



Vision with Reasoning (Created by Fareed Khan)

To solve this, we will build a two-stage pipeline that using a powerful and expensive vision model for the initial heavy lifting, followed by a smaller, cost-effective reasoning model to handle the logical refinement.

Here's the breakdown of our vision pipeline:



Vision Pipeline (Created by Fareed Khan)

1. We first set up the pipeline by selecting our models: a powerful **Vision Model** for OCR, a smaller **Reasoning Model** for logic, and a top-tier **QA Agent** for evaluation, all while defining a strict Pydantic schema to structure our final output.
2. The process begins by feeding the form image to the **Vision Model**, whose only job is to perform a high-fidelity OCR, transcribing text literally and carefully noting any visual ambiguities in its raw JSON output.
3. Next, the raw, ambiguous JSON is passed to the tool-using **Reasoning Model**, which acts as a data validation expert, using tools like web search to resolve ambiguities and fill in missing fields.

transformation and flags any fields requiring human attention.

5. Finally, we consolidate all the operational metrics and quality scores into a comprehensive summary, giving us a complete, end-to-end view of the pipeline performance, from cost and latency to a final confidence score.

We can now start building this vision pipeline.

Setting Up the Two-Stage Component

As we done previously, we begin by configuring our environment. We will select a powerful multimodal model for the initial OCR, a cheaper model for the tool-using refinement stage, and our top-tier model for the final quality evaluation.

Copy

```
from pydantic import BaseModel, Field
from IPython.display import display, Markdown, Image

# --- LLM Configuration for the Vision Pipeline ---
MODEL_VISION = "google/gemma-3-27b-it"    # A powerful multimodal model for OCR
MODEL_REASONING = "Qwen/Qwen3-14B"      # A smaller, cheaper model for refineme
MODEL_EVALUATE = "Qwen/Qwen3-235B-A22B"    # A powerful model for accurate eval
```

Before we begin, we must define our target data structure. Using Pydantic schemas acts as a strict contract for our LLMs, ensuring that even with messy visual input, the final output is clean, structured, and predictable.

Copy

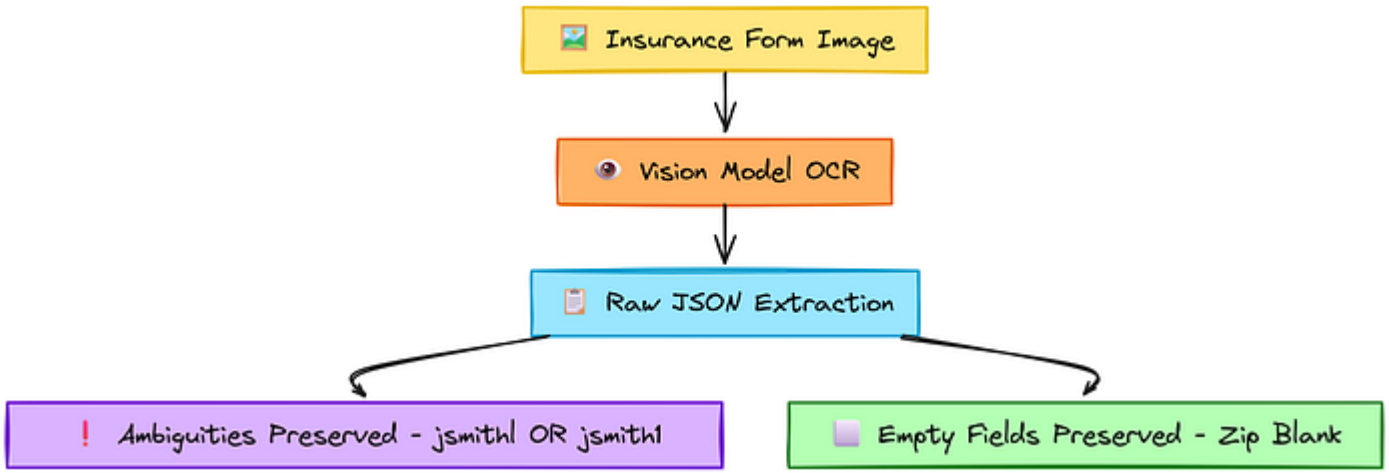
```
# Contact info
class PersonContact(BaseModel):
    # Stores applicant or co-applicant contact information
    name: str; home_phone: str; work_phone: str; cell_phone: str; email: str

# Address info
class Address(BaseModel):
    # Stores address details (risk or mailing)
    street: str; city: str; state: str; zip: str; county: str

# Dwelling details
class DwellingDetails(BaseModel):
    # Stores property details used for insurance risk assessment
    coverage_a_limit: str; companion_policy_expiration_date: str
    occupancy_of_dwelling: str; type_of_policy: str; unrepaired_structural_dama
    construction_type: str; roof_type: str; foundation_type: str
    has_post_and_pier_or_post_and_beam_foundation: bool; cripple_walls: bool
    number_of_stories: str; living_space_over_garage: bool; number_of_chimneys:
    square_footage: str; year_of_construction: str
    anchored_to_foundation: bool; water_heater_secured: bool
```

```
class InsuranceFormData(BaseModel):
    # Represents the complete insurance form with applicant, addresses, insurer
    applicant: PersonContact; co_applicant: PersonContact
    risk_address: Address; mailing_address_if_different_than_risk_address: Addr
    participating_insurer: str; companion_policy_number: str
    dwelling_details: DwellingDetails; effective_date: str; expiration_date: st

# Extract JSON from text
def parse_json_from_response(text: str) -> Dict[str, Any]:
    # Attempts to extract a JSON object from a response text (inside ```json ..
    m = re.search(r'```(?:json)?\s*({.*?})\s*```', text, re.S)
    j = m.group(1) if m else text[text.find('{'):text.rfind('}')+1]
    try:
        return json.loads(j)  # Return parsed JSON if valid
    except:
        logging.warning(f"Bad JSON: {text}")  # Log bad JSON
        return {"raw_text": text}  # Fallback to raw text
```

This schema is our blueprint. The goal of the entire pipeline is to take an image and populate this Pydantic model with clean, validated data.

High-Fidelity OCR with a Vision Model

In the first stage, we present an image of a hand-filled insurance form to our vision-capable LLM. The key here is the prompt, we don't ask the model to be



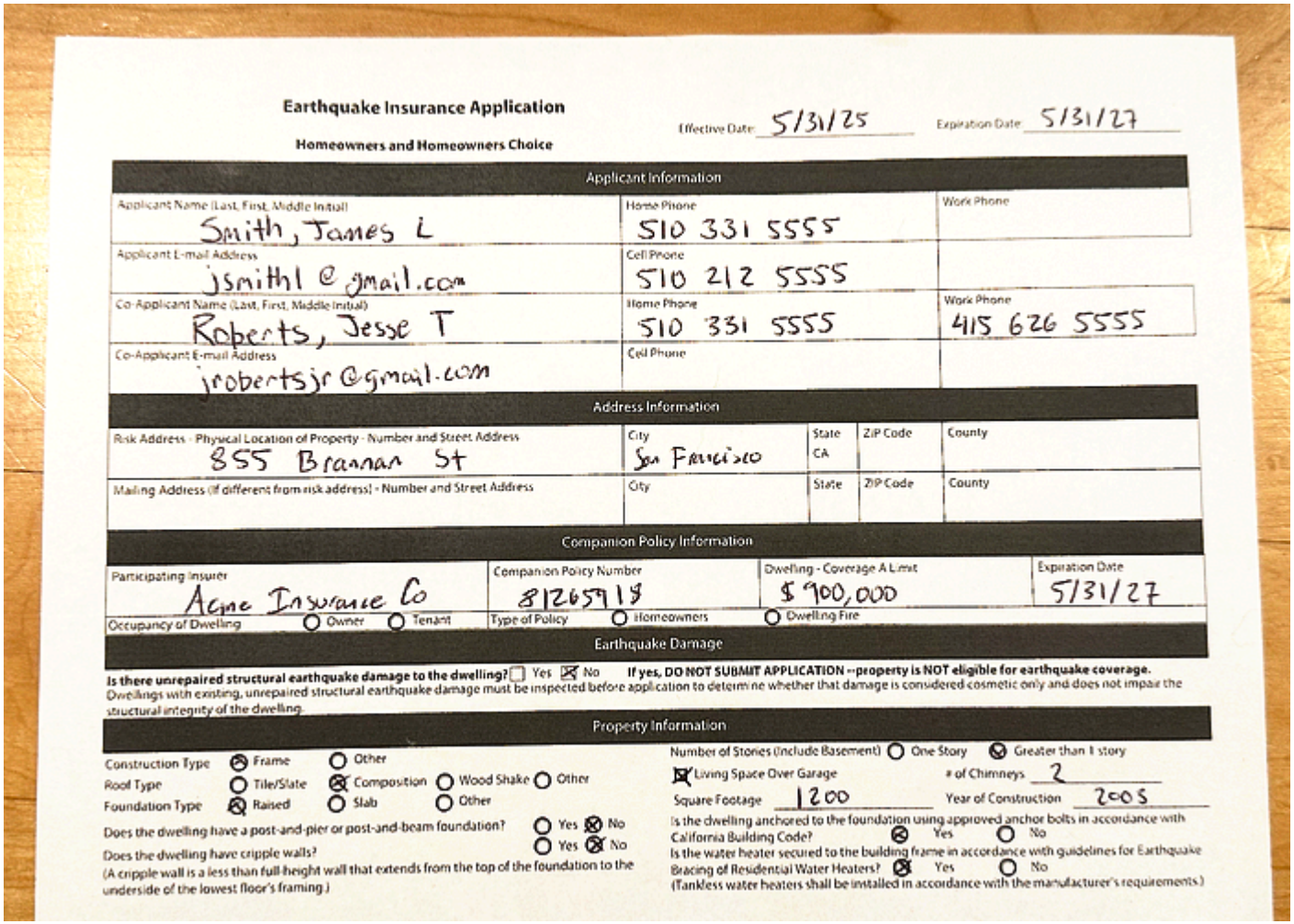
OCR with Vision (Creatd by Fareed Khan)

This is a important design choice. We want the model to capture any ambiguities it sees (e.g., if a character could be an "l" or a "1") and explicitly note them in the output.

> This prevents the vision model from making potentially incorrect guesses and provides a clean, raw extraction for our more specialized reasoning agent in the next stage.

Let's look at our source image:

```
# The image of the form we want to process
FORM_IMAGE_URL = "https://drive.usercontent.google.com/download?id=1-tZ526AW3mX
display(Image(url=FORM_IMAGE_URL, width=600))
```



Earthquake Insurance Application

Homeowners and Homeowners Choice

Effective Date: 5/31/25 Expiration Date: 5/31/27

Applicant Information

Applicant Name (Last, First, Middle Initial)	Home Phone	Work Phone
Smith, James L	510 331 5555	
Applicant E-mail Address	Cell Phone	
jsmith1@gmail.com	510 212 5555	
Co-Applicant Name (Last, First, Middle Initial)	Home Phone	Work Phone
Roberts, Jesse T	510 331 5555	415 626 5555
Co-Applicant E-mail Address	Cell Phone	
jrobertsjr@gmail.com		

Address Information

Risk Address - Physical Location of Property - Number and Street Address	City	State	ZIP Code	County
855 Brannan St	San Francisco	CA		
Mailing Address (if different from risk address) - Number and Street Address	City	State	ZIP Code	County

Companion Policy Information

Participating Insurer	Companion Policy Number	Dwelling - Coverage A Limit	Expiration Date
Acme Insurance Co	81265918	$ 900,000	5/31/27

Occupancy of Dwelling: ○ Owner ○ Tenant — Type of Policy: ○ Homeowners ○ Dwelling Fire

Earthquake Damage

Is there unrepaired structural earthquake damage to the dwelling? ☐ Yes ☒ No **If yes, DO NOT SUBMIT APPLICATION --property is NOT eligible for earthquake coverage.**
Dwellings with existing, unrepaired structural earthquake damage must be inspected before application to determine whether that damage is considered cosmetic only and does not impair the structural integrity of the dwelling.

Property Information

Construction Type: ⊗ Frame ○ Other
Roof Type: ○ Tile/Slate ⊗ Composition ○ Wood Shake ○ Other
Foundation Type: ⊗ Raised ○ Slab ○ Other

Does the dwelling have a post-and-pier or post-and-beam foundation? ○ Yes ⊗ No
Does the dwelling have cripple walls? ○ Yes ⊗ No
(A cripple wall is a less than full-height wall that extends from the top of the foundation to the underside of the lowest floor's framing.)

Number of Stories (Include Basement) ○ One Story ⊗ Greater than 1 story
☒ Living Space Over Garage # of Chimneys 2
Square Footage 1200 Year of Construction 2005
Is the dwelling anchored to the foundation using approved anchor bolts in accordance with California Building Code? ⊗ Yes ○ No
Is the water heater secured to the building frame in accordance with guidelines for Earthquake Bracing of Residential Water Heaters? ☒ Yes ○ No
(Tankless water heaters shall be installed in accordance with the manufacturer's requirements.)

ChatGPT generated AI Image (From OpenAI Cookbook)

```
def run_ocr_stage(image_url: str) -> Dict[str, Any]:
    """
    Uses a powerful vision model to perform a literal OCR of the form image.
    """
    ocr_prompt = f"""You are an expert at processing insurance forms. OCR the c

    IMPORTANT INSTRUCTIONS:
    1. Fill out the fields as literally and exactly as possible.
    2. If a field is blank, leave the string empty ('').
    3. If a character is ambiguous (e.g., 'l' or '1'), include all possibilitie
    4. Do NOT infer or guess any information not explicitly written on the form

    messages = [
        {"role": "user", "content": [
            {"type": "text", "text": ocr_prompt},
            {"type": "image_url", "image_url": {"url": image_url}}
        ]}
    ]

    response = client.chat.completions.create(model=MODEL_VISION, messages=mess
    return parse_json_from_response(response.choices[0].message.content)

# Run the first stage of the pipeline
stage1_output = run_ocr_stage(FORM_IMAGE_URL)
display(Markdown("### Stage 1 OCR Output (with ambiguities):"))
display(stage1_output)
```

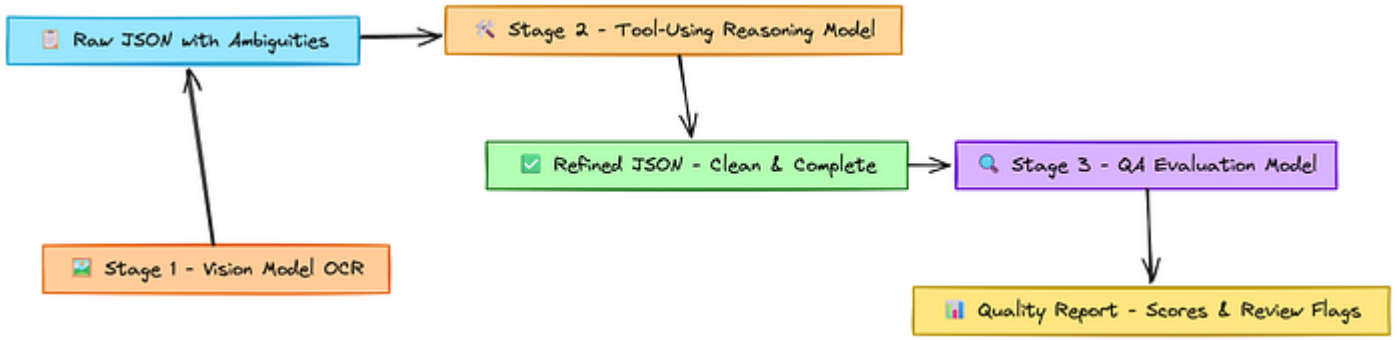
it encounters. This "dumb but accurate" extraction is exactly what we need for a robust system.

```
Copy

### Stage 1 OCR Output (with ambiguities):
{'applicant': {'name': 'Smith, James L', 'email': 'jsmithl@gmail.com OR jsmith1
 'risk_address': {'street': '855 Brannan St', 'city': 'San Francisco', 'state':
 'dwelling_details': {'construction_type': 'Wood Shake OR Other'}
 ...
}
```

The output is exactly what we hoped for. The vision model has successfully transcribed the form while preserving the ambiguities it found. Notice the `email` field contains both `jsmithl` and `jsmith1`, and the `zip` and `county` fields are correctly left blank. This raw, ambiguous data is the perfect input for our next stage.

Refinement with a Tool-Using Reasoning Model

Now, we pass the raw JSON from Stage 1 to our reasoning agent, powered by the cheaper `MODEL_REASONING`. This agent's job is to act as a data validation expert.



Tool using Reason (Created by Fareed Khan)

To ground our agent reasoning, we provide it with tools that can interact with the outside world. Here, we mock a simple email validator and a web search tool.

Copy

```
# --- Mock Tools for Stage 2 ---
def validate_email(email: str) -> bool:
    """Mock function to validate an email. For this demo, only 'jsmithl@gmail.c
    return email == "jsmithl@gmail.com"

def search_web(query: str) -> str:
    """Mock function for web search to find missing address details."""
    if "855 Brannan St" in query:
        return "The full address is 855 Brannan St, San Francisco, CA 94103. Th
    return "No information found."
```

```
# (The get_tool_manifest function is defined similarly to the previous use case
```

With its tools ready, the refinement agent can now take the raw OCR data and intelligently clean it up.

Copy

```
def run_refinement_stage(ocr_data: Dict[str, Any]) -> Dict[str, Any]:
    """
    Uses a smaller, tool-using LLM to clean, validate, and complete the raw OCR
    """
    refinement_prompt = """You are a data validation expert. You have been give
    1. Examine each field for ambiguities (e.g., text with ' OR '). Use tools t
    2. Identify missing information (e.g., empty strings for `zip`). Use tools
    3. If the mailing address is empty, assume it is the same as the risk addre
    4. Return the final, cleaned JSON object that conforms to the Pydantic sche

    messages = [
        {"role": "system", "content": refinement_prompt},
        {"role": "user", "content": f"Here is the raw OCR data to refine:\n\n{j
    ]

    # This loop allows the agent to make multiple tool calls if needed
    for i in range(5):
        response = client.chat.completions.create(model=MODEL_REASONING, messag
        msg = response.choices[0].message
        messages.append(msg)
```

```
            return parse_json_from_response(msg.content)

        # Execute tool calls and append results to the conversation
        for tool_call in msg.tool_calls:
            function_name = tool_call.function.name
            args = json.loads(tool_call.function.arguments)
            result = TOOL_DISPATCHER[function_name](**args)
            messages.append({"role": "tool", "tool_call_id": tool_call.id, "cor

    return {"error": "Exceeded tool call limit."}

# Run the second stage of the pipeline
stage2_output = run_refinement_stage(stage1_output)
display(Markdown("### Stage 2 Refined Output:"))
display(stage2_output)
```

In the second stage, we are creating the same iterative tool-calling loop as our Co-Scientist. This will allow the agent to reason, act, and observe, such as by first attempting to validate one email, and if it fails, trying the other.

Copy

```
### Stage 2 Refined Output:
{'applicant': {'name': 'Smith, James L', 'email': 'jsmithl@gmail.com'},
 'risk_address': {'street': '855 Brannan St', 'city': 'San Francisco', 'state':
 'mailing_address_if_different_than_risk_address': {'street': '855 Brannan St',
```

The transformation is good. The reasoning agent successfully used its tools to resolve the ambiguous email to `jsmithl@gmail.com` and to perform a web search to fill in the missing zip code and county.

It also correctly inferred that the mailing address was the same as the risk address. The result is a clean, complete, and validated data object.

> But how can we be sure of the quality of this transformation? As our final component, we will code an **Automated Quality Assurance (QA) Agent.**

This agent, based on our LLM act as an evaluator `MODEL_EVALUATE` , will work as a auditor. It compares the raw, ambiguous output from Stage 1 with the clean, refined output from Stage 2. Its job is to provide a detailed report, scoring the process on multiple dimensions and, most importantly, flagging any fields that may still require human review.

Copy

```
def evaluate_extraction_quality(raw_data: Dict, final_data: Dict) -> Dict:
    """
```

```
    eval_prompt = f"""You are a Quality Assurance expert. Compare the "RAW OCR

    RAW OCR DATA:
    {json.dumps(raw_data, indent=2)}

    FINAL REFINED DATA:
    {json.dumps(final_data, indent=2)}

    Provide your evaluation with the following structure:
    {{
      "field_accuracy_score": {{ "score": float (0.0-1.0), "justification": "As
      "inference_quality_score": {{ "score": float (0.0-1.0), "justification":
      "completeness_score": {{ "score": float (0.0-1.0), "justification": "Calc
      "overall_confidence_score": {{ "score": float (0.0-1.0), "justification"
      "fields_requiring_human_review": ["list", "of", "field_names", "that stil
    }}
    """

    messages = [{"role": "user", "content": eval_prompt}]
    response = client.chat.completions.create(model=MODEL_EVALUATE, messages=me
    return parse_json_from_response(response.choices[0].message.content)

# Run the final evaluation stage
quality_assessment = evaluate_extraction_quality(stage1_output, stage2_output)
display(Markdown("### Stage 3 Automated Quality Report:"))
display(quality_assessment)
```

and generating actionable feedback, transforming our system from a black box into a transparent and observable process.

```
                                                    Copy

### Stage 3 Automated Quality Report:
{'field_accuracy_score': {'score': 0.95, 'justification': 'The final data accu
 'inference_quality_score': {'score': 0.7, 'justification': 'The model success
 'completeness_score': {'score': 1.0, 'justification': 'All fields in the fina
 'overall_confidence_score': {'score': 0.85, 'justification': 'The final data
 'fields_requiring_human_review': ['mailing_address_if_different_than_risk_addr
```

This report is highly insightful. It gives high scores for accuracy and completeness but correctly lowers the `inference_quality_score` because the logic for filling the mailing address was an assumption.

It also flags the `mailing_address` fields as requiring human review, providing a clear, actionable signal to a downstream process or human operator.

Analyzing the Results

As with our previous use cases, we'll finish by consolidating all our operational and quality metrics into a final summary. This gives us a complete, end-to-end

Copy

```
# Define model pricing for this use case
model_prices_per_million_tokens = {
    "google/gemma-3-27b-it": {"input": 0.10, "output": 0.30},
    "Qwen/Qwen3-14B": {"input": 0.08, "output": 0.24},
    "Qwen/Qwen3-235B-A22B": {"input": 0.20, "output": 0.60}
}

# Create the final summary DataFrame
if metrics_log and quality_assessment:
    def get_score(assessment, key):
        # Helper to safely extract scores
        return assessment.get(key, {}).get('score', 0.0)

    summary_data = {
        'Metric': [
            'Total Latency (s)', 'Total Cost (USD)', 'Total Tokens',
            '--- Quality Scores (AI) ---', 'Field Accuracy Score', 'Inference (
            'Completeness Score', '**Overall Confidence Score**', 'Fields for H
        ],
        'Value': [
            f"{df_metrics['latency_s'].sum():.2f}", f"${df_metrics['cost_usd'].
            f"{df_metrics['total_tokens'].sum():,}", '---',
            f"{get_score(quality_assessment, 'field_accuracy_score'):.2f}",
            f"{get_score(quality_assessment, 'inference_quality_score'):.2f}",
            f"{get_score(quality_assessment, 'completeness_score'):.2f}",
            f"**{get_score(quality_assessment, 'overall_confidence_score'):.2f}
            ', '.join(quality_assessment.get('fields_requiring_human_review',
```

```
df_summary = pd.DataFrame(summary_data).set_index("Metric")
display(Markdown("### Final Run Summary"))
display(df_summary)
```

Metric	Value
Total Latency (s)	165.94
Total Cost (USD)	$0.003793
Total Tokens	12,376
Field Accuracy Score	0.95
Inference Quality Score	0.70
Completeness Score	1.00
Overall Confidence Score	0.85
Fields for Human Review	mailing_address_if_different_than_risk_address...

comparison.md hosted with ❤️ by GitHub view raw

We processed a complex visual form in under three minutes for less than a cent. The automated QA gives us a high overall confidence of **0.85**, while also providing a clear, actionable list of fields that need a second look.

Analyzing Our Findings

Across three distinct and challenging use cases, we have moved beyond abstract benchmarks and evaluated open-source LLMs. Let's take a look at our findings.

- **For Broad, High-Throughput Tasks (Routing, Ideation):** Smaller, faster models like `Qwen/Qwen3-4B-fast` and `Llama-3.1-8B-Instruct` proved to be the champions. They are incredibly cost-effective and their speed is essential for tasks that require processing large amounts of data or generating many parallel outputs. Their lower reasoning power is not a major drawback in these roles.
- **For Focused, Logical Tasks (Safety, Refinement):** Mid-size models like `Qwen/Qwen3-14B` hit the sweet spot. They possess strong instruction-following and tool-use capabilities, making them perfect for specialized, logical tasks that don't require the deep understanding of a flagship model.
- **For Deep, High-Stakes Reasoning (Synthesis, Critique, Evaluation):** This is where the largest models, like `Qwen/Qwen3-235B-A22B`, `Llama-3.3-70B-Instruct`, and `DeepSeek-V3`, are indispensable. Their superior reasoning, synthesis, and evaluation capabilities justify their higher cost and latency when the quality of the final output is paramount.
- **For Multimodal Perception (OCR):** Specialized vision-capable models like `gemma-3-27b-it` are a necessity. This demonstrates that for certain tasks, the

Key Takeaways

There is a lot of information in this blog, let's quickly summarize the most important points.

- **No single best LLM**: The most effective systems combine multiple models, using cheaper ones for scale and expensive ones for precision.
- **Role-based selection**: Assign models to roles like router, synthesizer, or judge, rather than seeking one model for everything.
- **Escalation pattern**: Begin with fast, low-cost models to structure or filter problems, then escalate promising cases to stronger models for deeper analysis.
- **Automated evaluation**: Reliable pipelines require continuous measurement, using top-tier LLMs as judges to score quality and uncover weaknesses.
- **Observability**: Monitoring cost, latency, and qualitative metrics in a unified view is critical for optimization, debugging, and demonstrating value.

> In case you enjoy this blog, feel free to follow me on Medium I only write here.